

Natural Language Processing

Post training



David O'Neil

Schedule

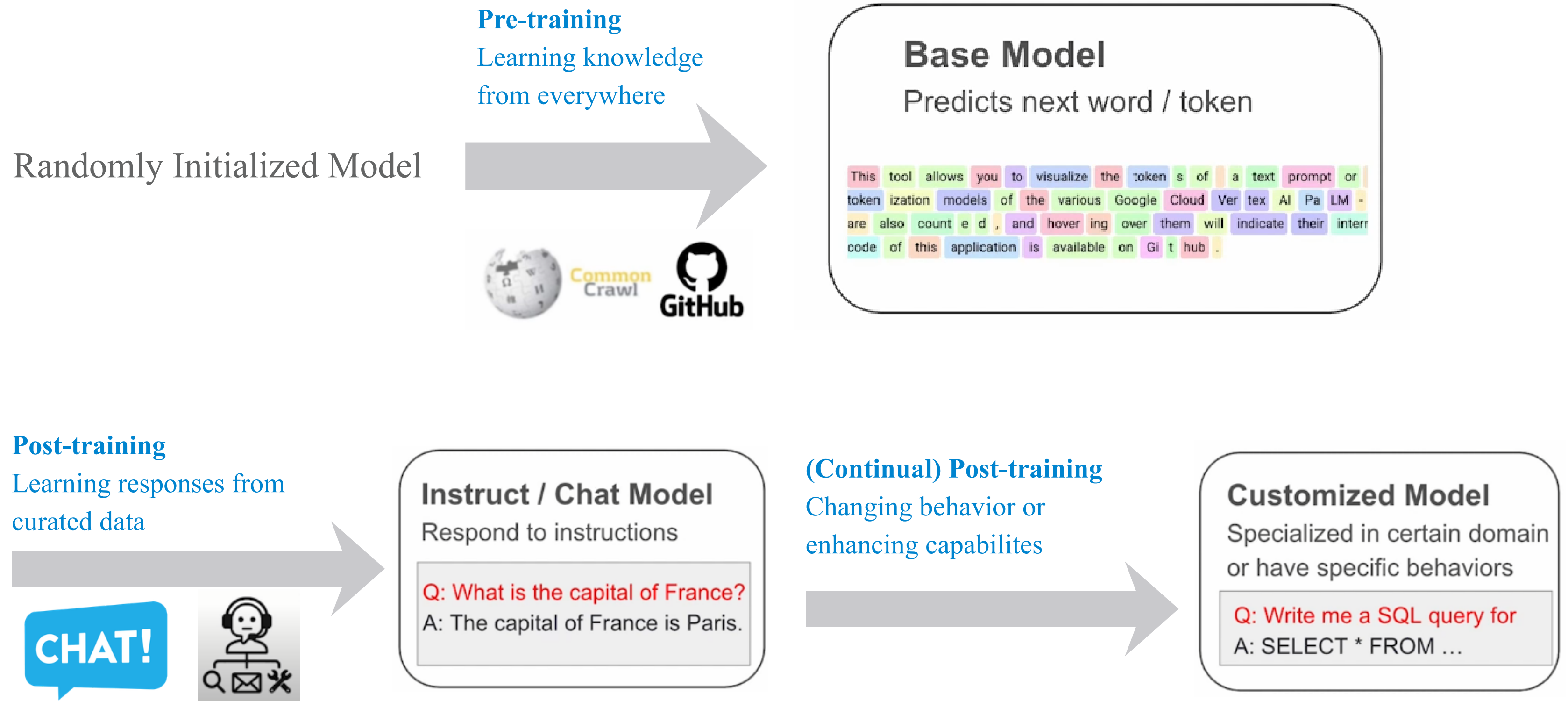
- Post-Training: Overview
- Supervised Fine-Tuning (SFT)
- Instruction Tuning
 - Dados de Demonstração e Instruction–Response
- Curadoria de Dados de Pós-Treino **
- Dados Sintéticos para Pós-Treino **
- SFT Loss
 - Cross-Entropy / Next-Token Prediction Loss
- Full-Parameter Fine-Tuning (Full SFT)
- Parameter-Efficient Fine-Tuning (PEFT)
 - LoRA — Low-Rank Adaptation
- Full Fine-Tuning vs. LoRA
- Reinforcement Learning from Human Feedback (RLHF)
 - Reward Modeling
 - Human Feedback vs. AI Feedback
- Proximal Policy Optimization (PPO)
 - Policy Objective
- Direct Preference Optimization (DPO)
 - DPO Loss
- Reasoning Models **
- Reinforcement Learning for Reasoning **
- GRPO — Group Relative Policy Optimization
- Avaliação de LLMs **
- Benchmarking **
- Safety e Failure Modes **



Post-Training: Overview



Post-Training: Overview

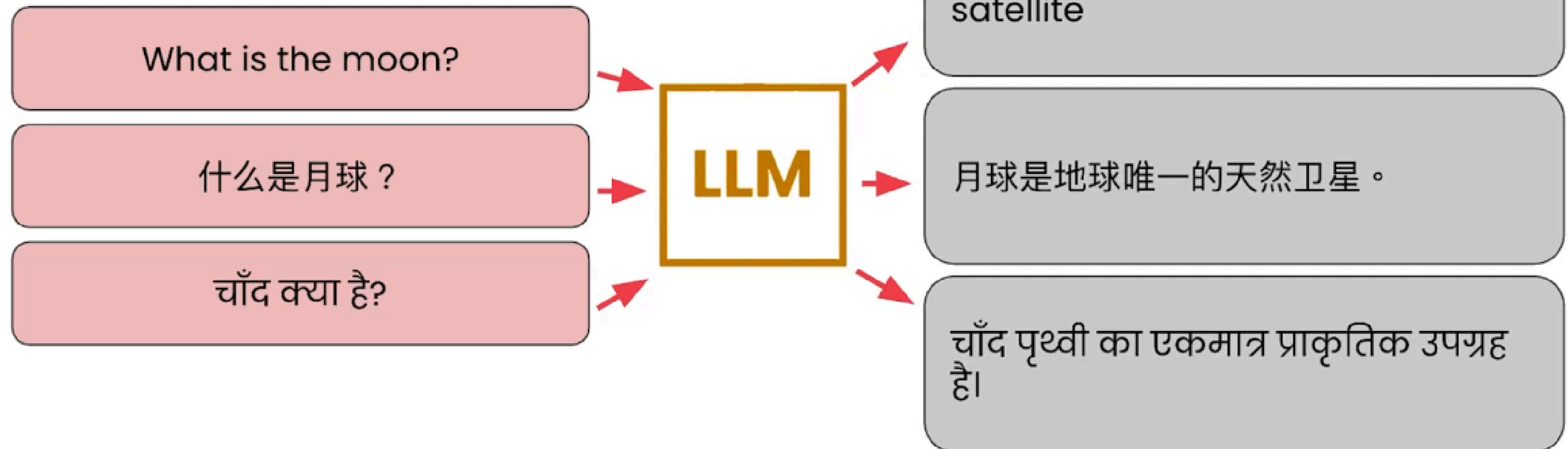




Post-Training: Overview

Pre-training

Add new languages



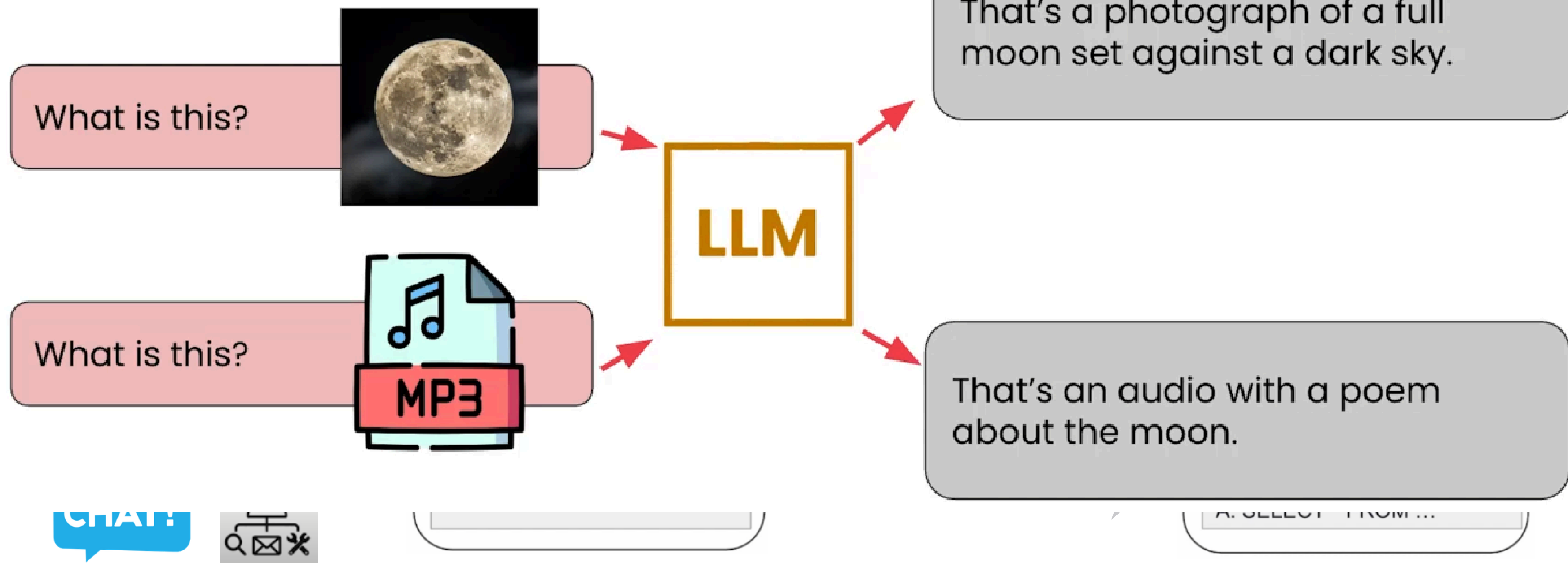
A: SELECT * FROM ...



Post-Training: Overview

Pre-training

Add multimodality



Post-Training: Overview

*** <https://arxiv.org/abs/2309.12307>

Feito por SFT →



*** <https://arxiv.org/abs/2402.10171>

Feito por Continual Pretraining→

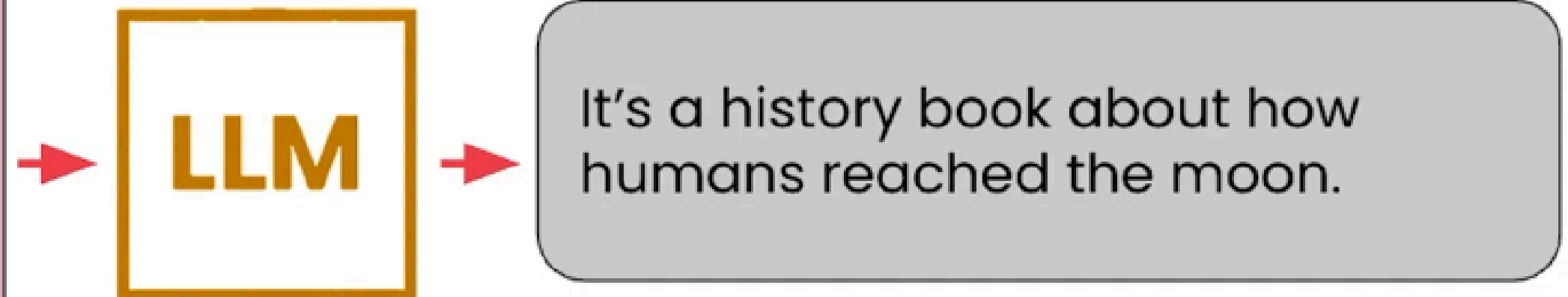


Increase context length limits

Context:

„President John F. Kennedy addressed a joint session of Congress. He laid down a bold, seemingly impossible challenge: “I believe that this nation should commit itself to achieving the goal, before this decade is out, of landing a man on the Moon and returning him safely to the Earth.” This declaration transformed a scattered series of projects into a singular, national purpose. It was a race against the clock and against a formidable rival, one that would require unprecedented technological innovation, immense financial investment, and the dedication of hundreds of thousands of people. This is the story of that race—a tale of tragedy and triumph, of ingenuity and human will, and of the single greatest technical achievement in history. Chapter I: Forging the Tools To fulfill Kennedy’s mandate, NASA needed to design a multi-faceted approach. They couldn’t just build one rocket; they had to create a family of rockets, spacecraft, and procedures that had never existed. The Apollo program was a system of interlocking parts, each one a technological marvel in its own right. The Saturn V Rocket: The Colossus of the Skies At the heart of the Apollo program was the Saturn V, the most powerful rocket ever built and a masterpiece of engineering. Standing over 360 feet tall and weighing more than 6 million pounds at launch, it was a multi-stage behemoth designed to propel the Apollo spacecraft out of Earth’s gravity and towards the moon. First Stage (S-IC): The monstrous first stage was powered by five F-1 engines, the most powerful liquid-fueled rocket engines ever developed. At liftoff, they generated a staggering 7.7 million pounds of thrust. The roar was so immense that it could be felt miles away. This stage burned for just 2.5 minutes, lifting the rocket to an altitude of 38 miles before it was jettisoned, falling into the Atlantic Ocean. Second Stage (S-II): After...

Question: What does this book talk about?



It's a history book about how humans reached the moon.



Supervised Fine-Tuning (SFT)

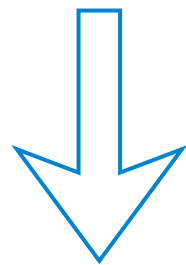
Methods Used During LLM Training

Pre-Training

(Unsupervised Learning)



>>2T tokens



"I like *cats*"

$$\min_{\pi} -\log \pi (I) -\log \pi (\text{like} \mid I) \\ -\log \pi (\text{cats} \mid \text{I like})$$

Post-Training (SFT)

(Supervised / Imitation Learning)

Labeled Prompt-Response
Pairs

Prompt: Explain LLM to me
Response: LLM is ...

~1K-1B tokens



$$\min_{\pi} -\log \pi (\text{Response} \mid \text{Prompt})$$

Instruction Tuning

*** <https://arxiv.org/abs/2308.10792v5>

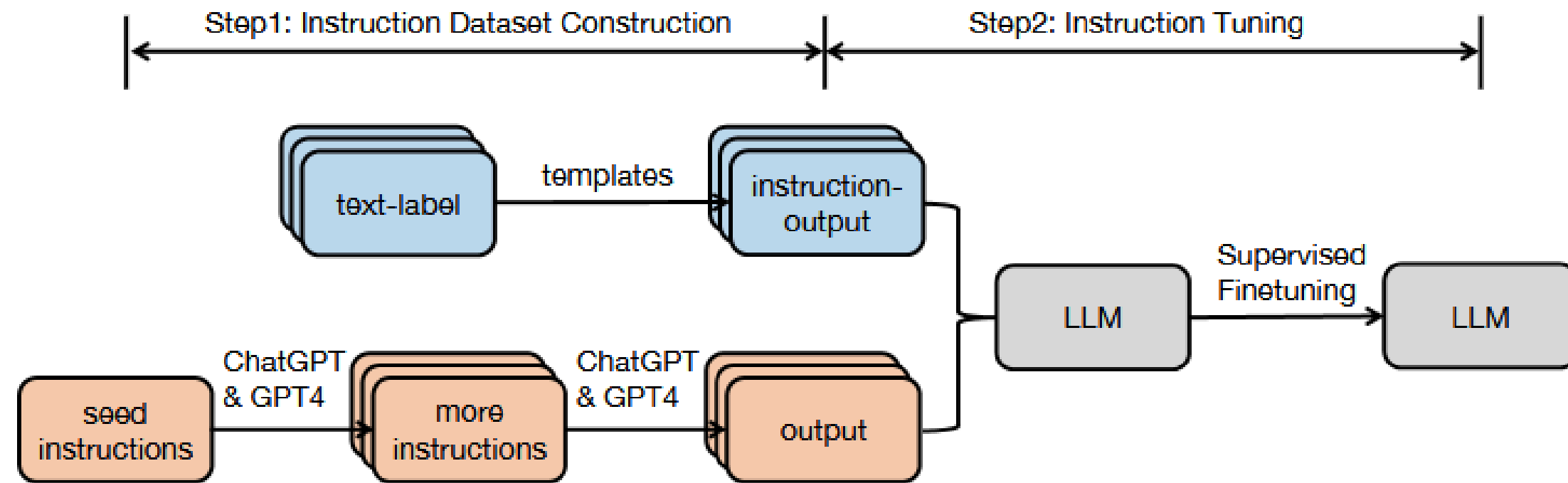
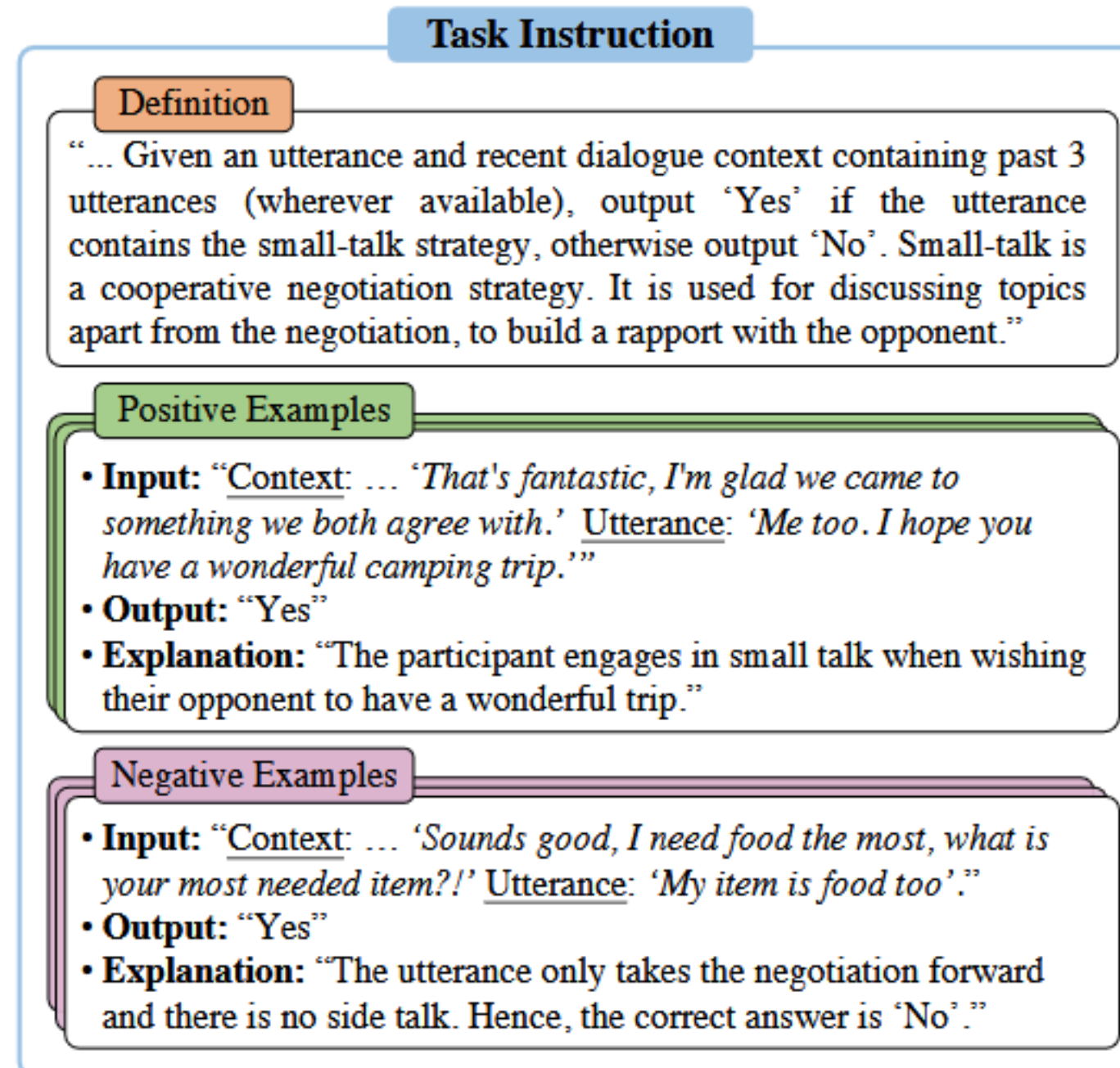


Figure 1: General pipeline of instruction tuning.

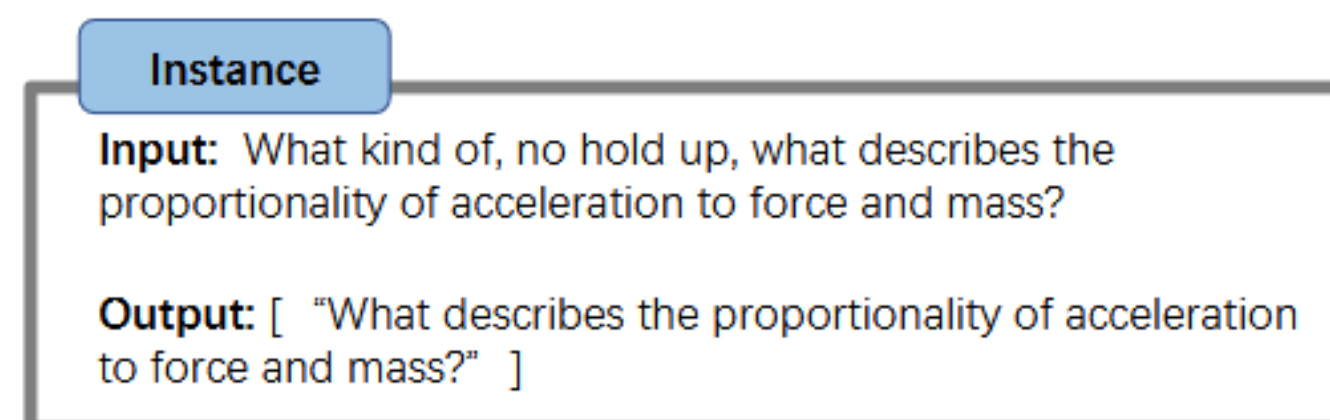
Instruction Tuning



*** <https://arxiv.org/abs/2308.10792v5>



(a) An example of INSTRUCTIONS in Super-Natural Instruction dataset.



(b) An example of INSTANCES in Super-Natural Instruction dataset.

Instruction Tuning

*** <https://arxiv.org/abs/2308.10792v5>



Instruction Type	Example
Open Q&A	Why do people like comedy movies?
Closed Q&A	Does outbreeding or inbreeding benefit the offspring more?
Information Extraction	Who was John Moses Browning?
Information Summarization	Please summarize what LinkedIn does.
Brainstorming	Give me some ideas to manage my manager.
Classification	Identify which animal species is alive or extinct: Palaeophis, Giant Tortoise
Creative writing	Write a short story about a person who discovers a hidden room in their house.

Instruction Tuning

*** <https://arxiv.org/abs/2308.10792v5>

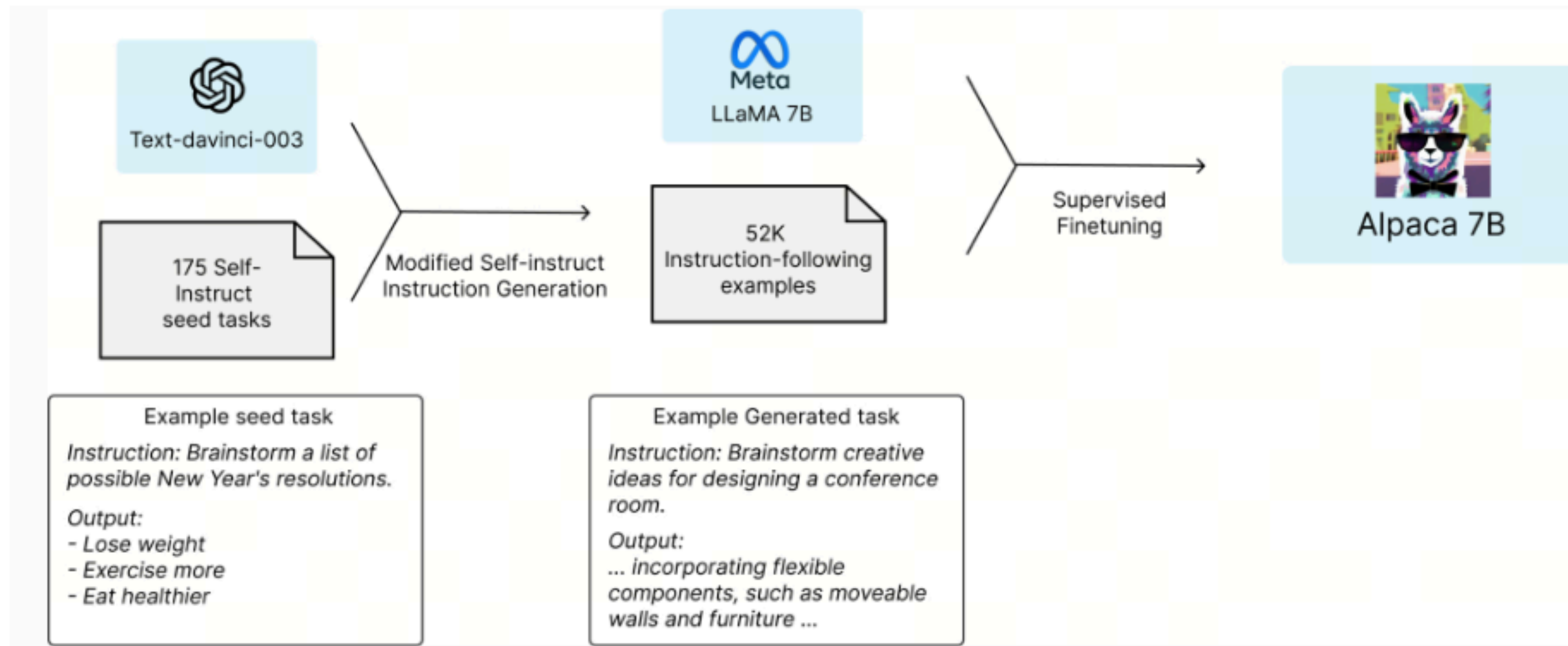
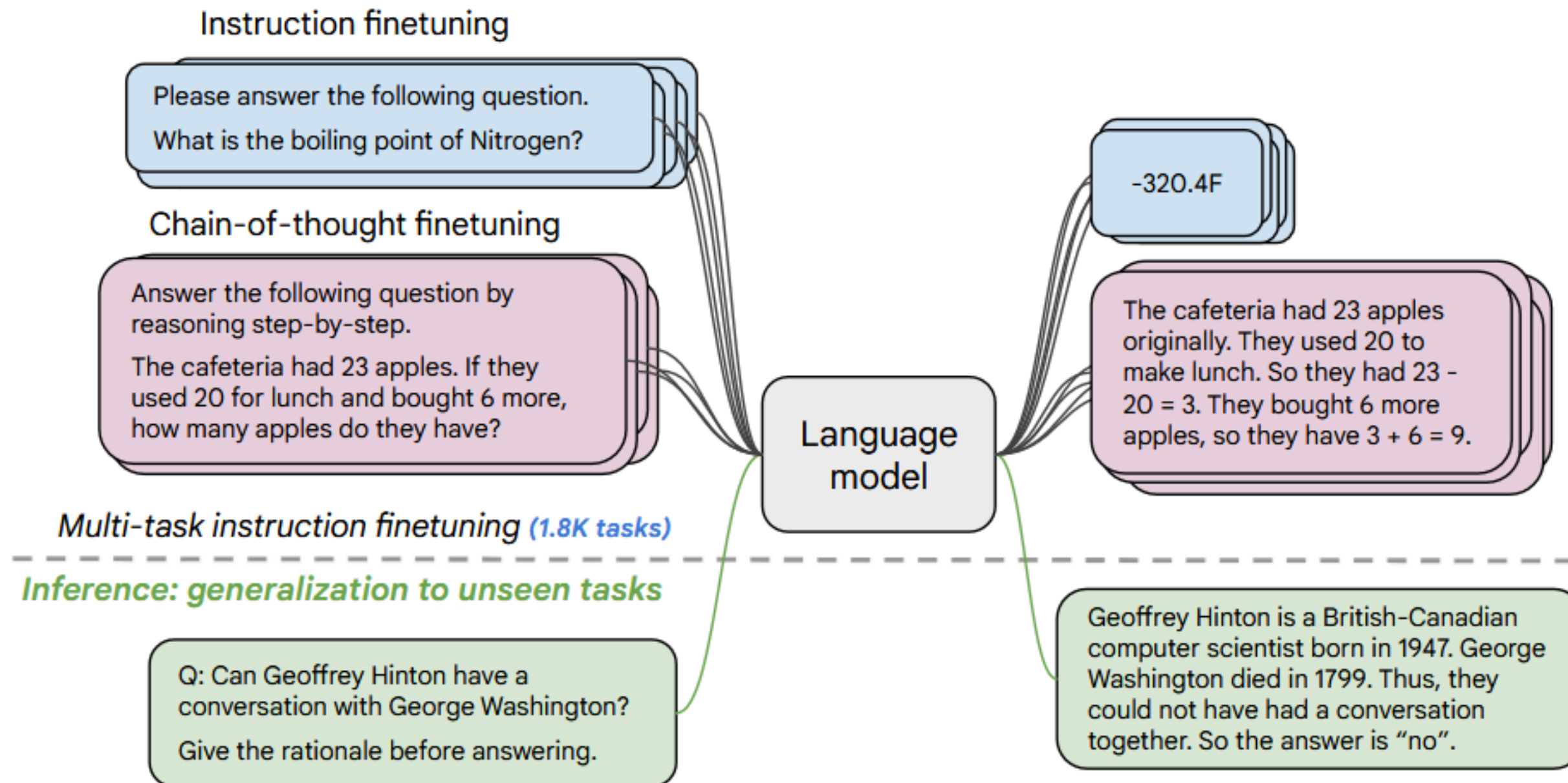


Figure 5: General pipeline of distillation for synthetic data generation. The figure is adapted from Taori et al. (2023a).

Instruction Tuning

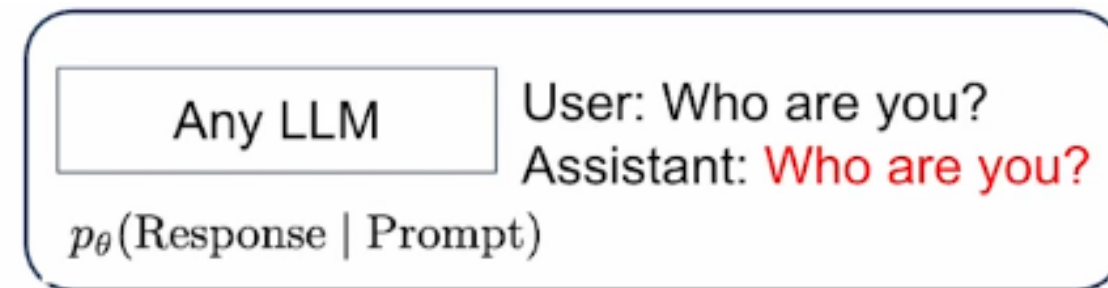
Dados de Demonstração e Instruction–Response

*** <https://arxiv.org/abs/2210.11416>



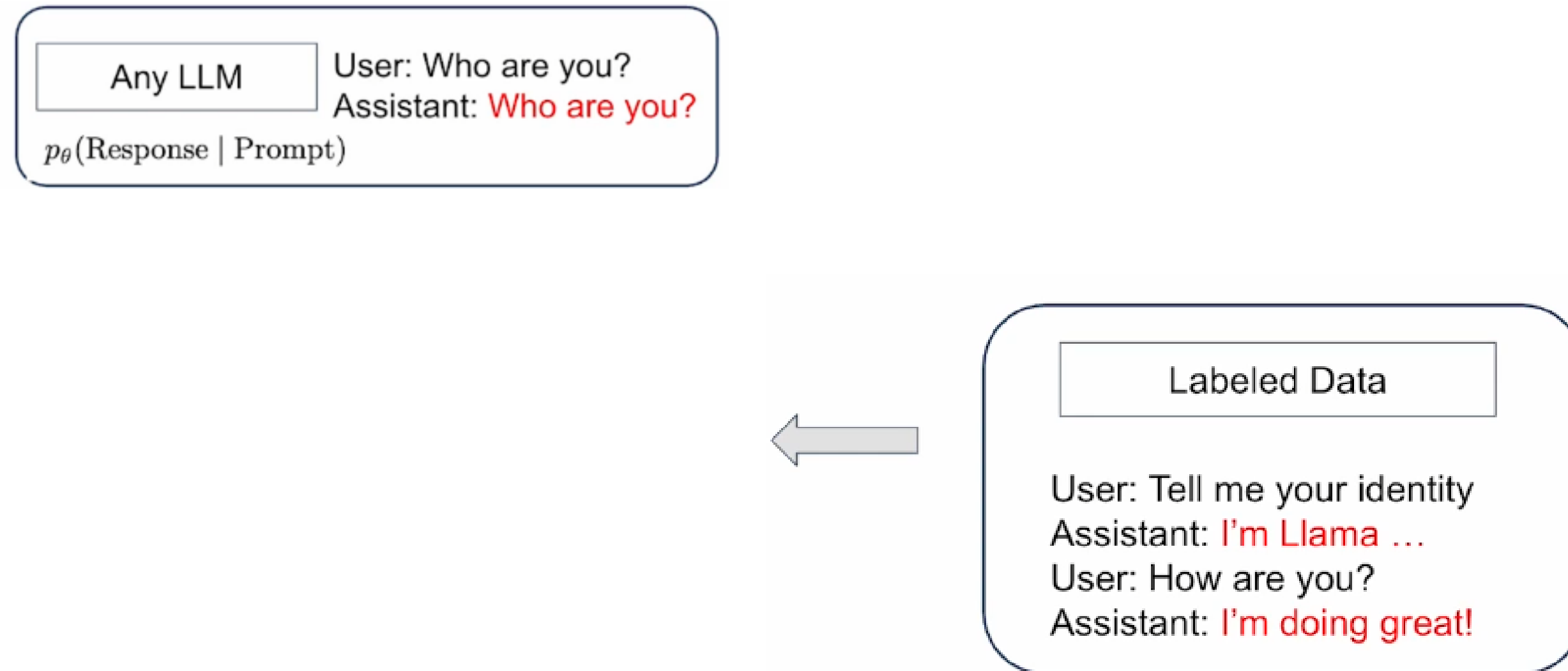


SFT Loss



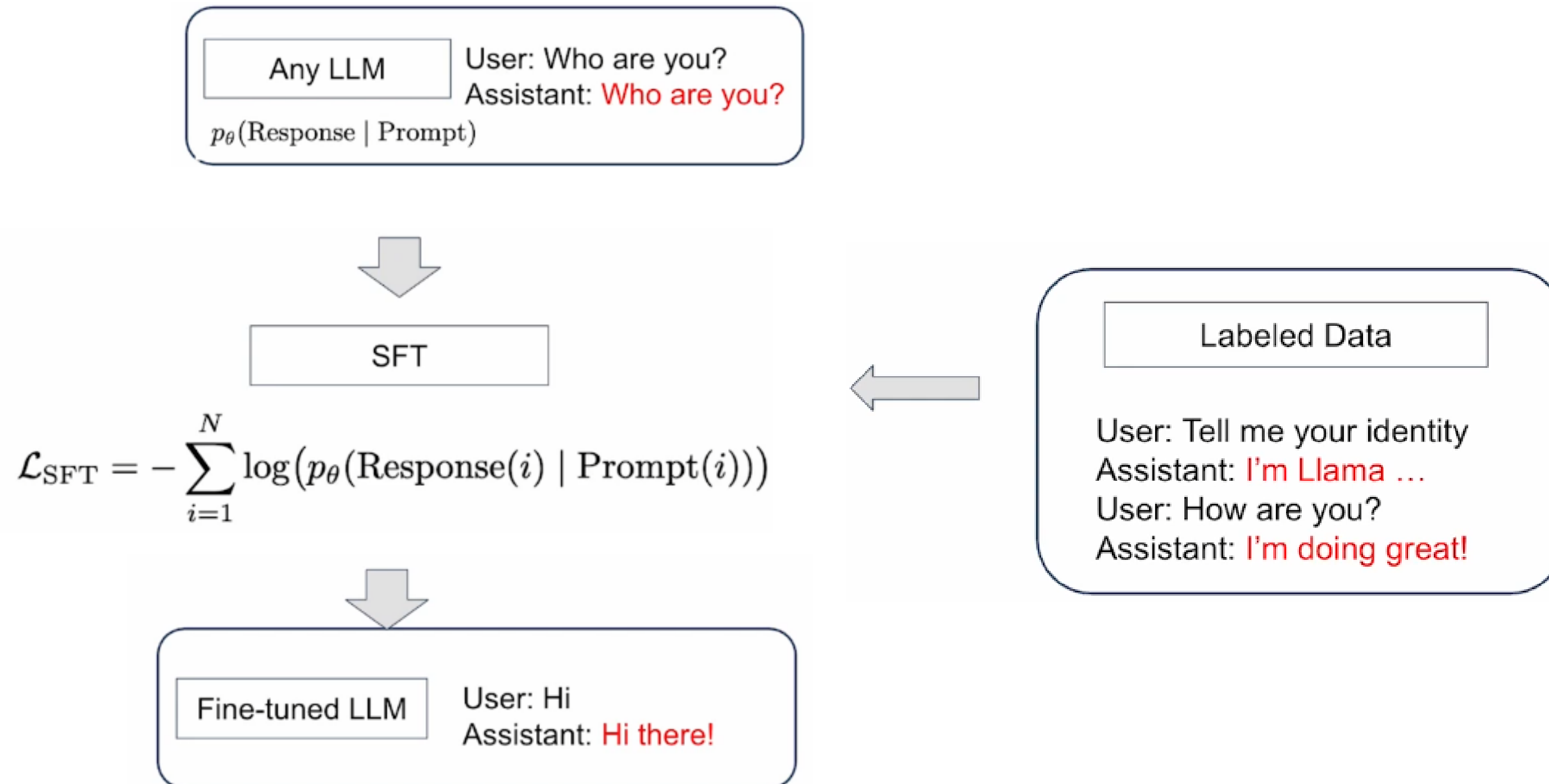


SFT Loss





SFT Loss





SFT Loss

SFT minimizes negative log likelihood for the responses (maximizes likelihood) with cross entropy loss:

$$\mathcal{L}_{\text{SFT}} = - \sum_{i=1}^N \log(p_{\theta}(\text{Response}(i) \mid \text{Prompt}(i)))$$

SFT: Imitating Example Responses



SFT Loss

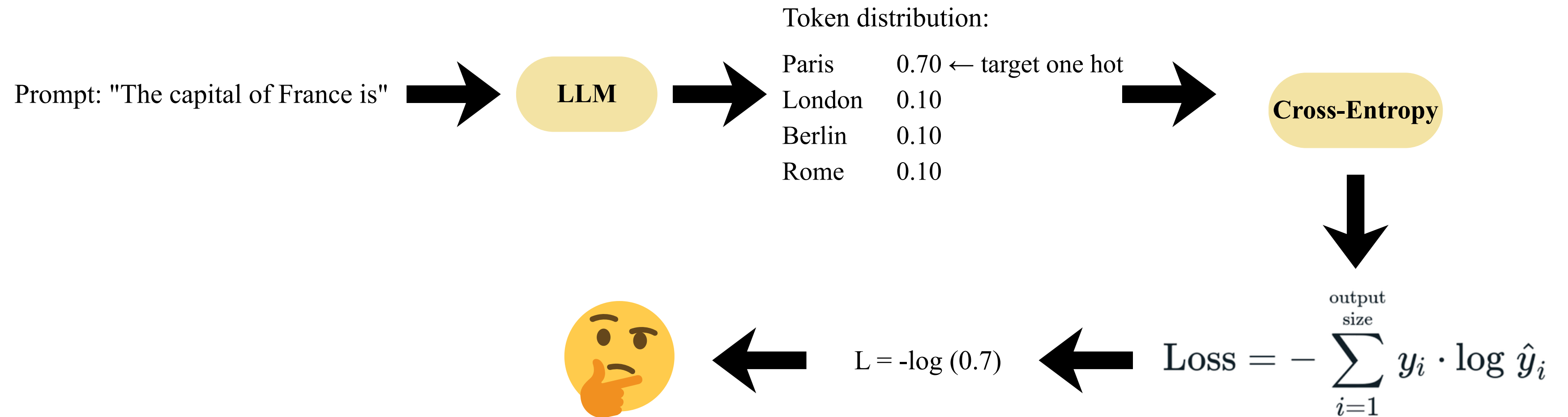
Cross-Entropy / Next-Token Prediction

$$\mathcal{L}_{\text{SFT}} = - \sum_{i=1}^N \log(p_{\theta}(\text{Response}(i) \mid \text{Prompt}(i)))$$

De onde vem esse “- log p” ?

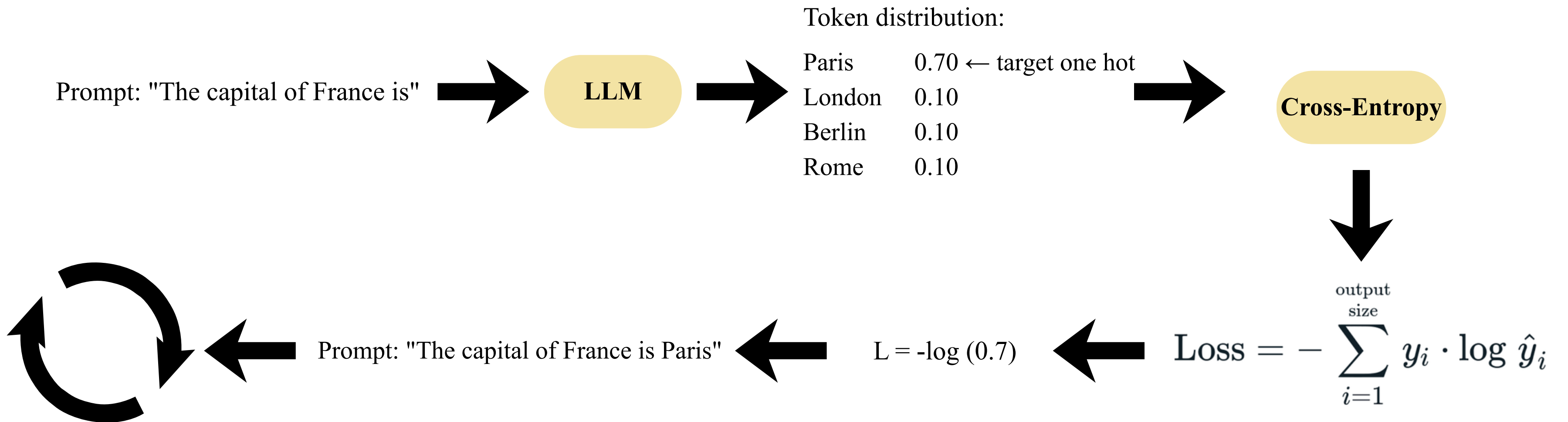
SFT Loss

Cross-Entropy / Next-Token Prediction



SFT Loss

Cross-Entropy / Next-Token Prediction



SFT Loss

Cross-Entropy / Next-Token Prediction

Next-Token Prediction

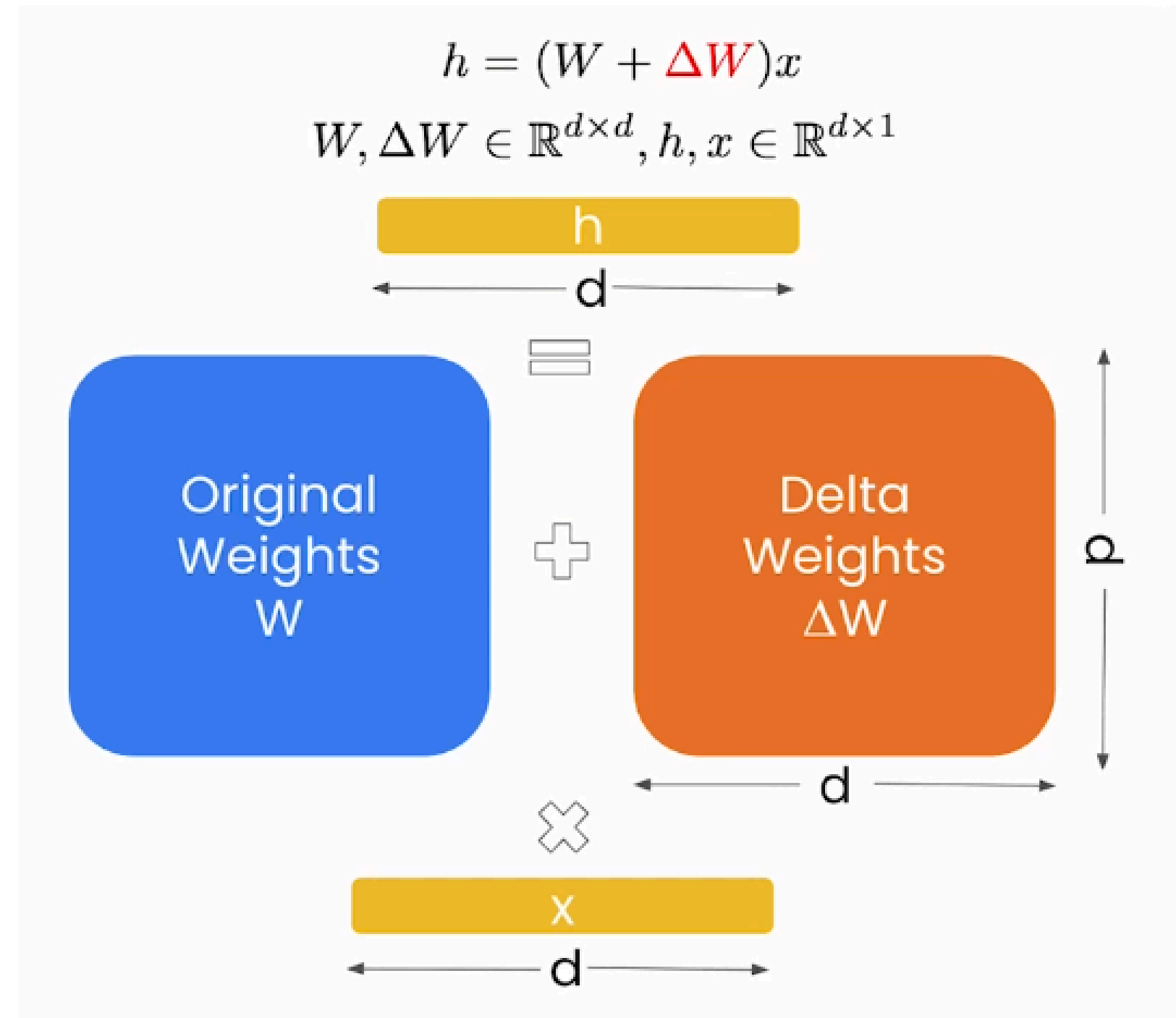


Cross-Entropy



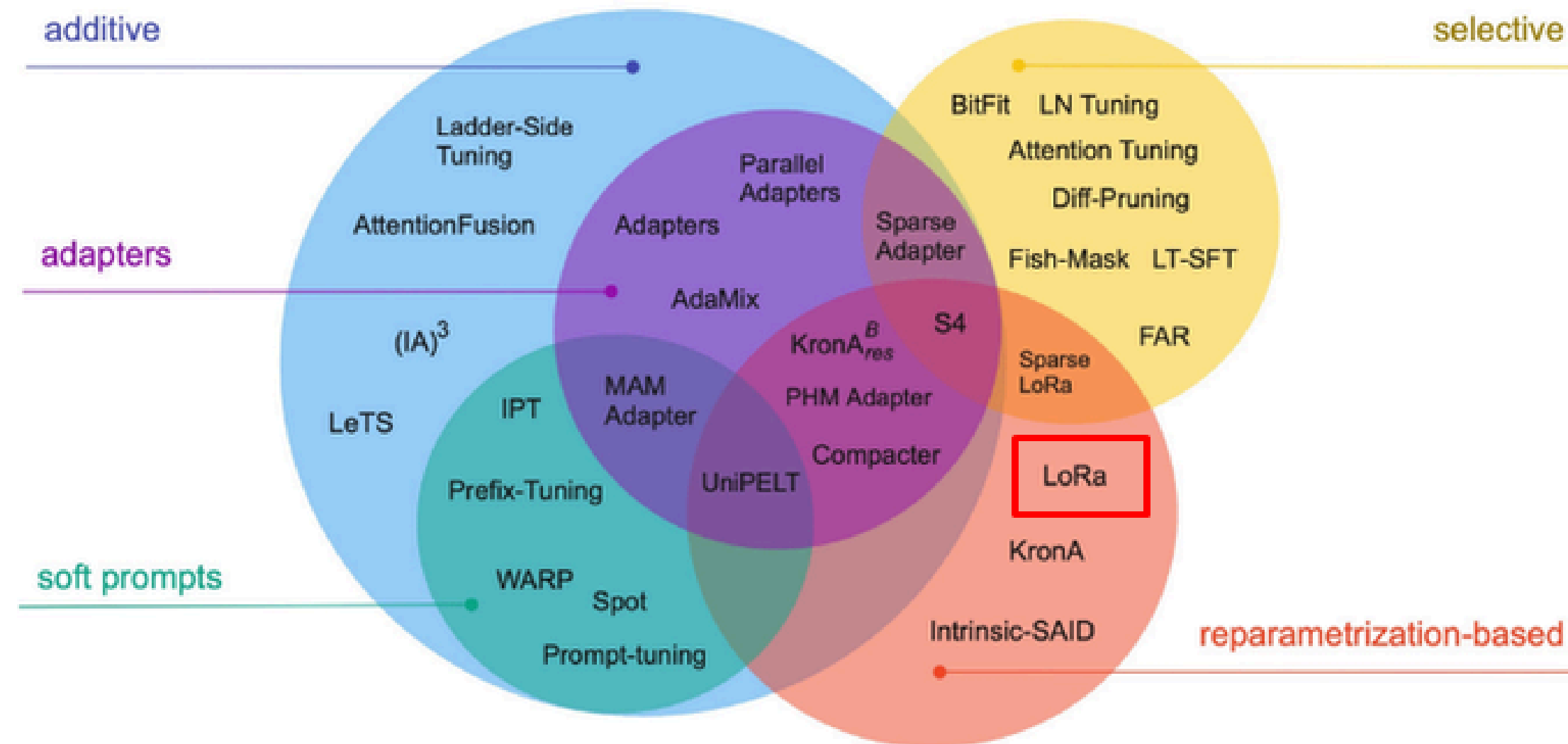


Full-Parameter Fine-Tuning (Full SFT)



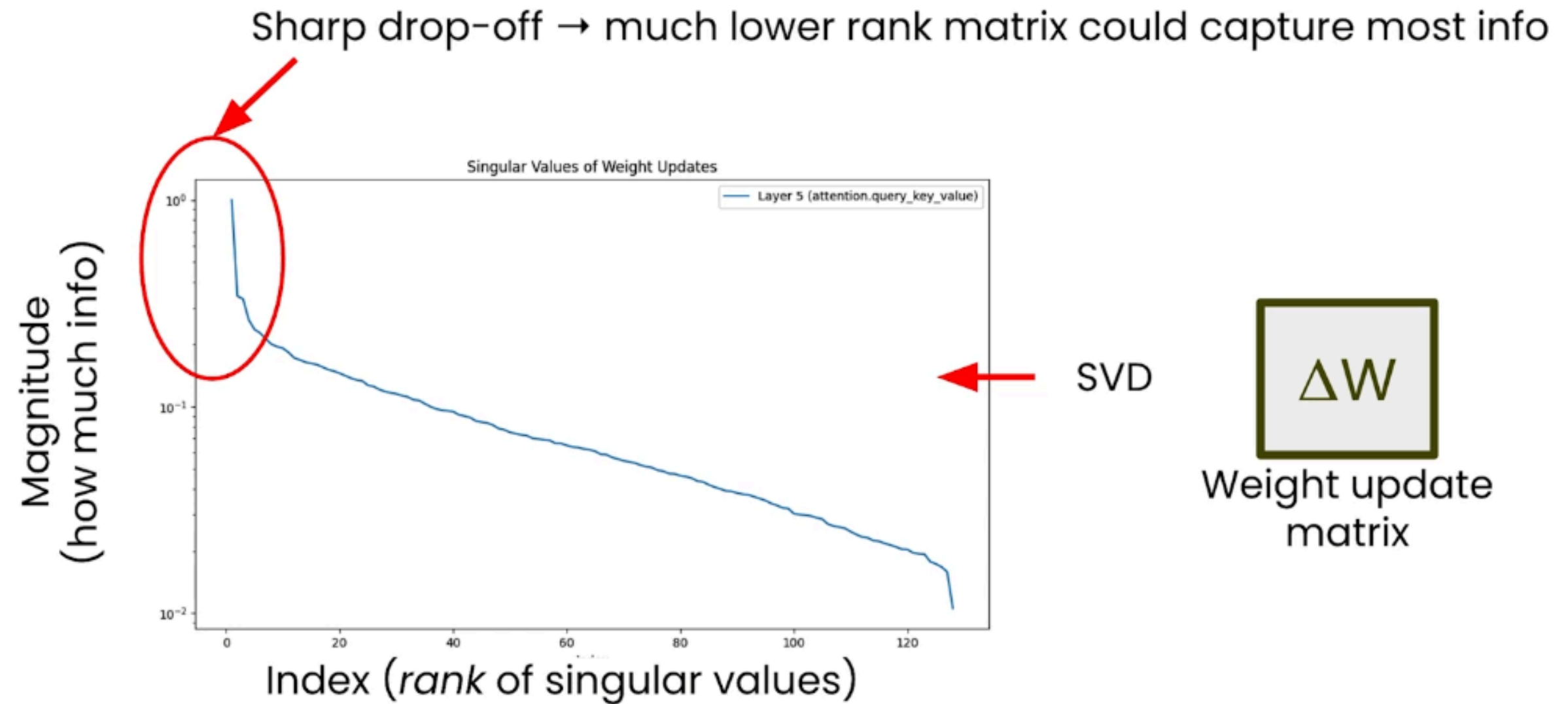
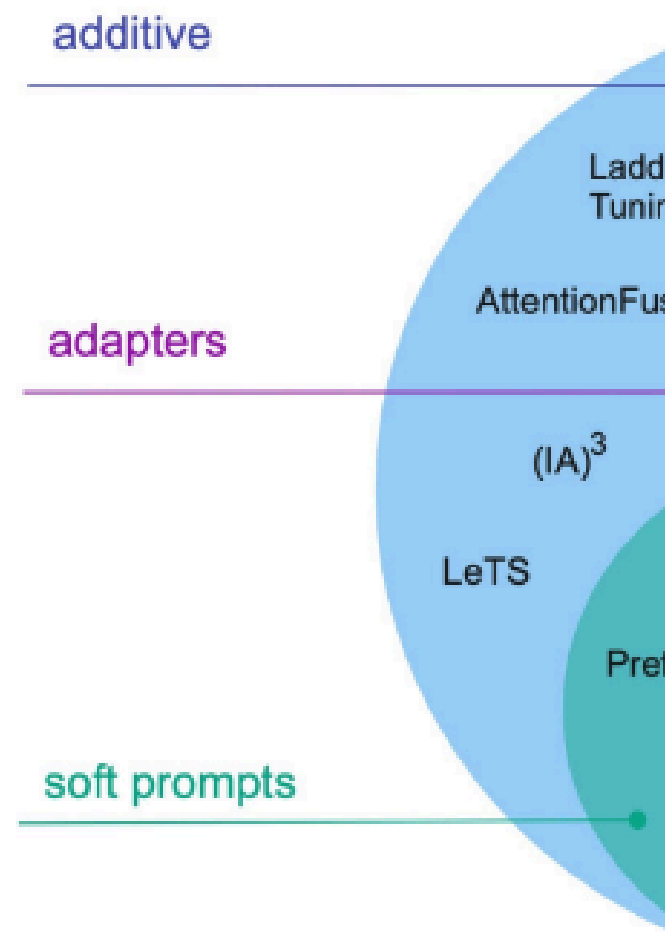


Parameter-Efficient Fine-Tuning (PEFT)

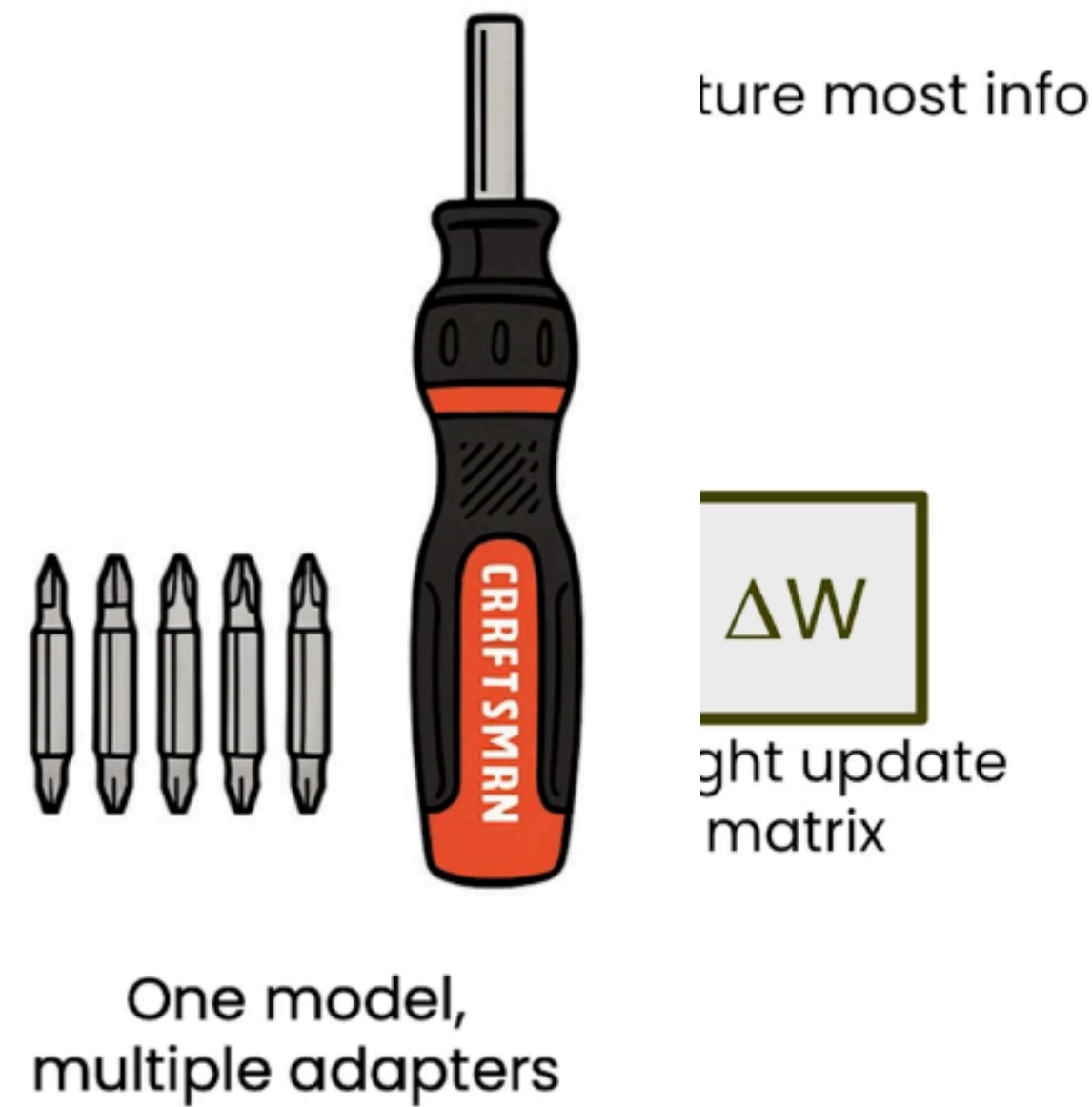
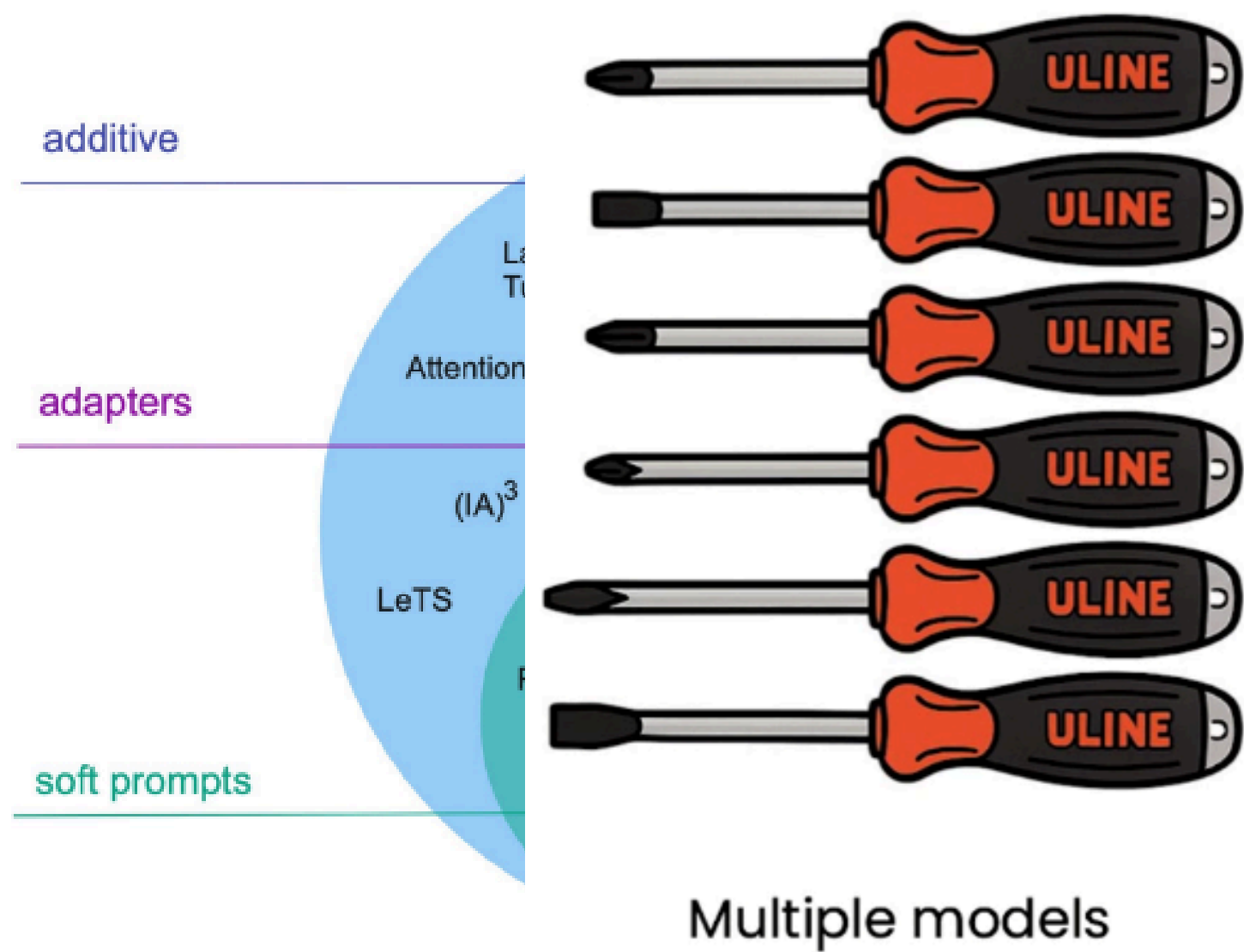




Parameter-Efficient Fine-Tuning (PEFT)



Parameter-Efficient Fine-Tuning (PEFT)

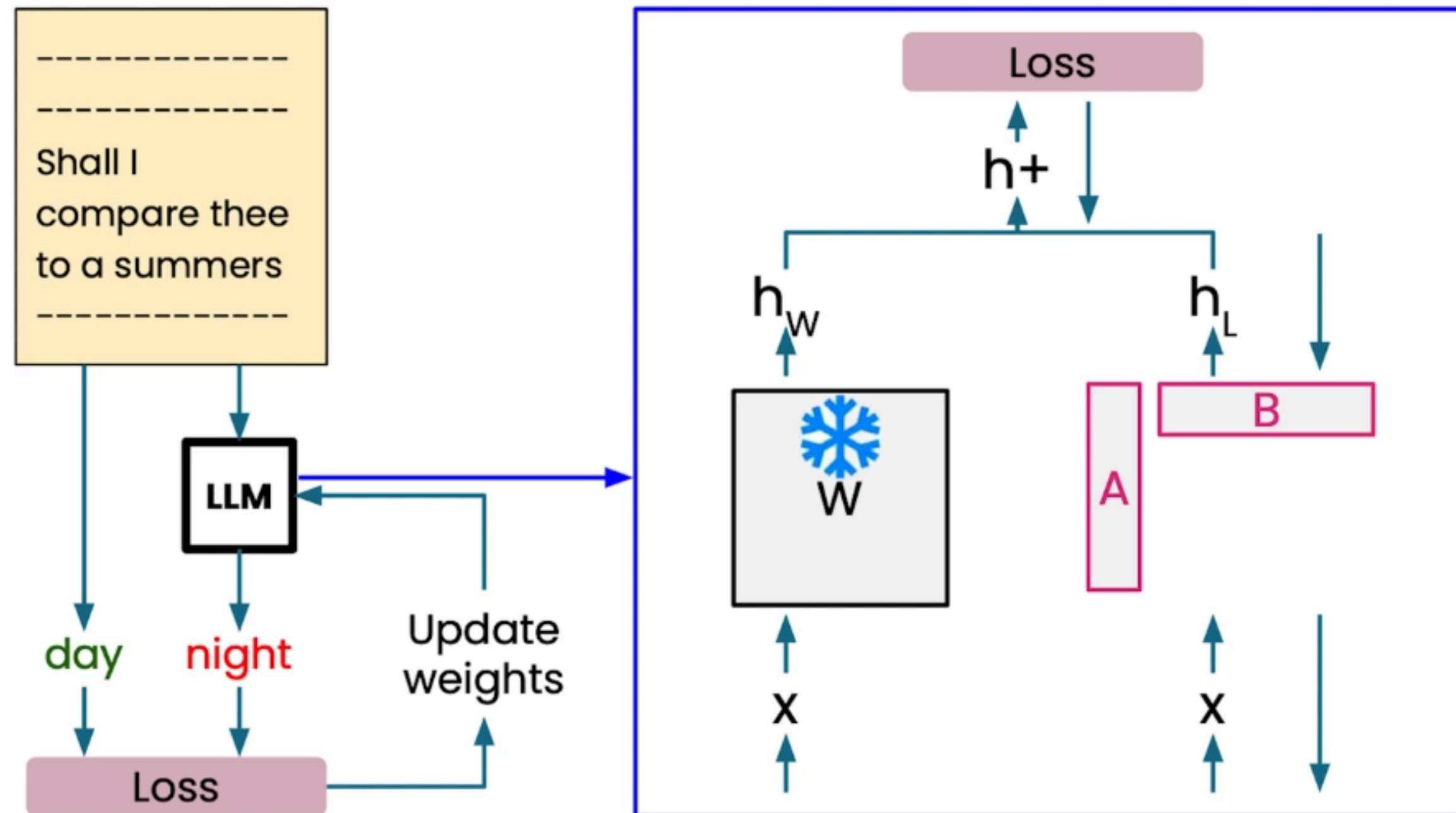




Parameter-Efficient Fine-Tuning (PEFT)

LoRA — Low-Rank Adaptation

*** <https://learn.deeplearning.ai/courses/fine-tuning-and-reinforcement-learning-for-llms-intro-to-post-training>





Parameter-Efficient Fine-Tuning (PEFT)

LoRA — Low-Rank Adaptation

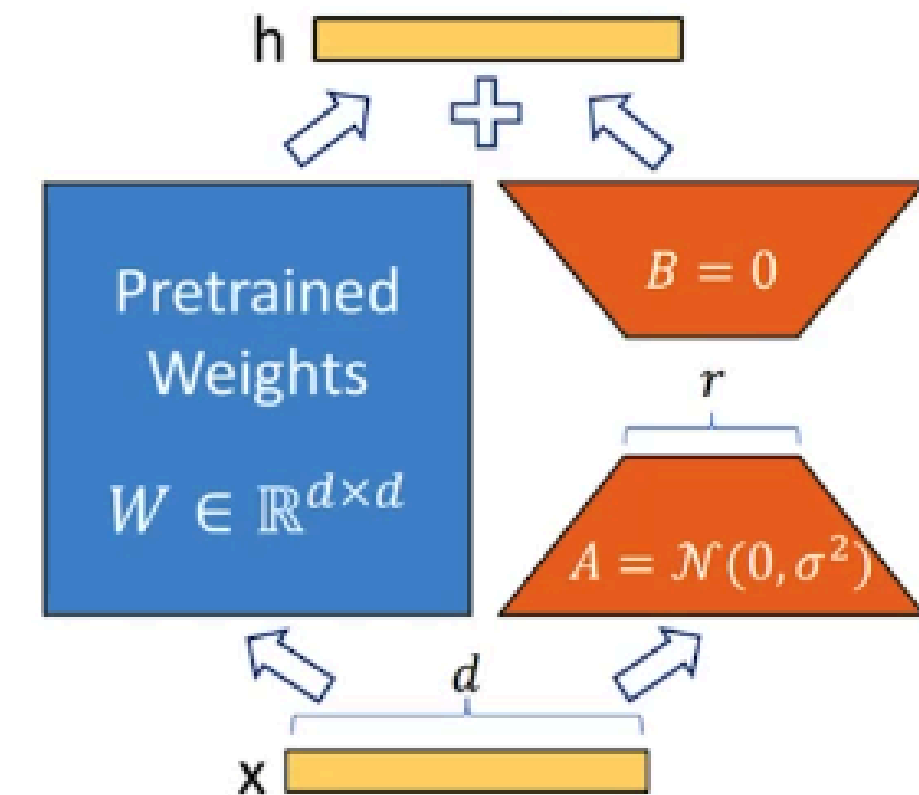
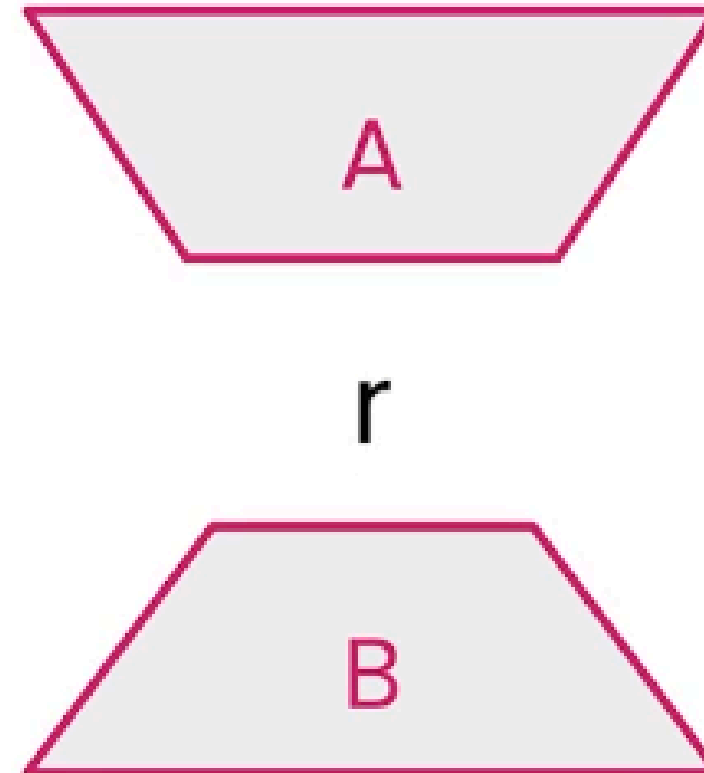
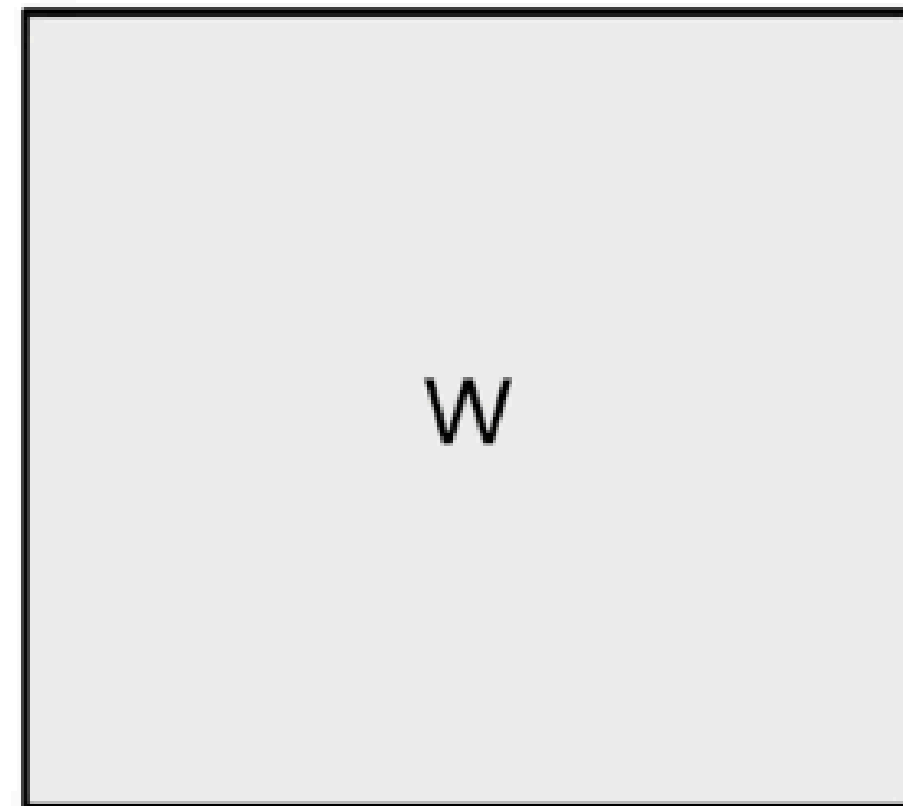


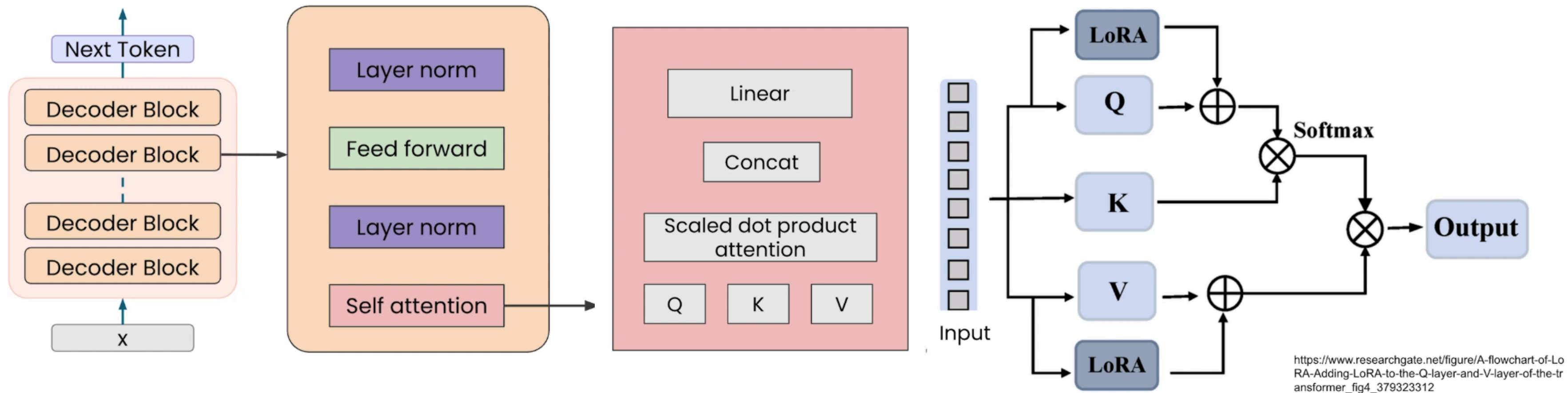
Figure 1: Our reparametrization. We only train A and B .

From Hu et al. 2021



Parameter-Efficient Fine-Tuning (PEFT)

LoRA — Low-Rank Adaptation



Parameter-Efficient Fine-Tuning (PEFT)

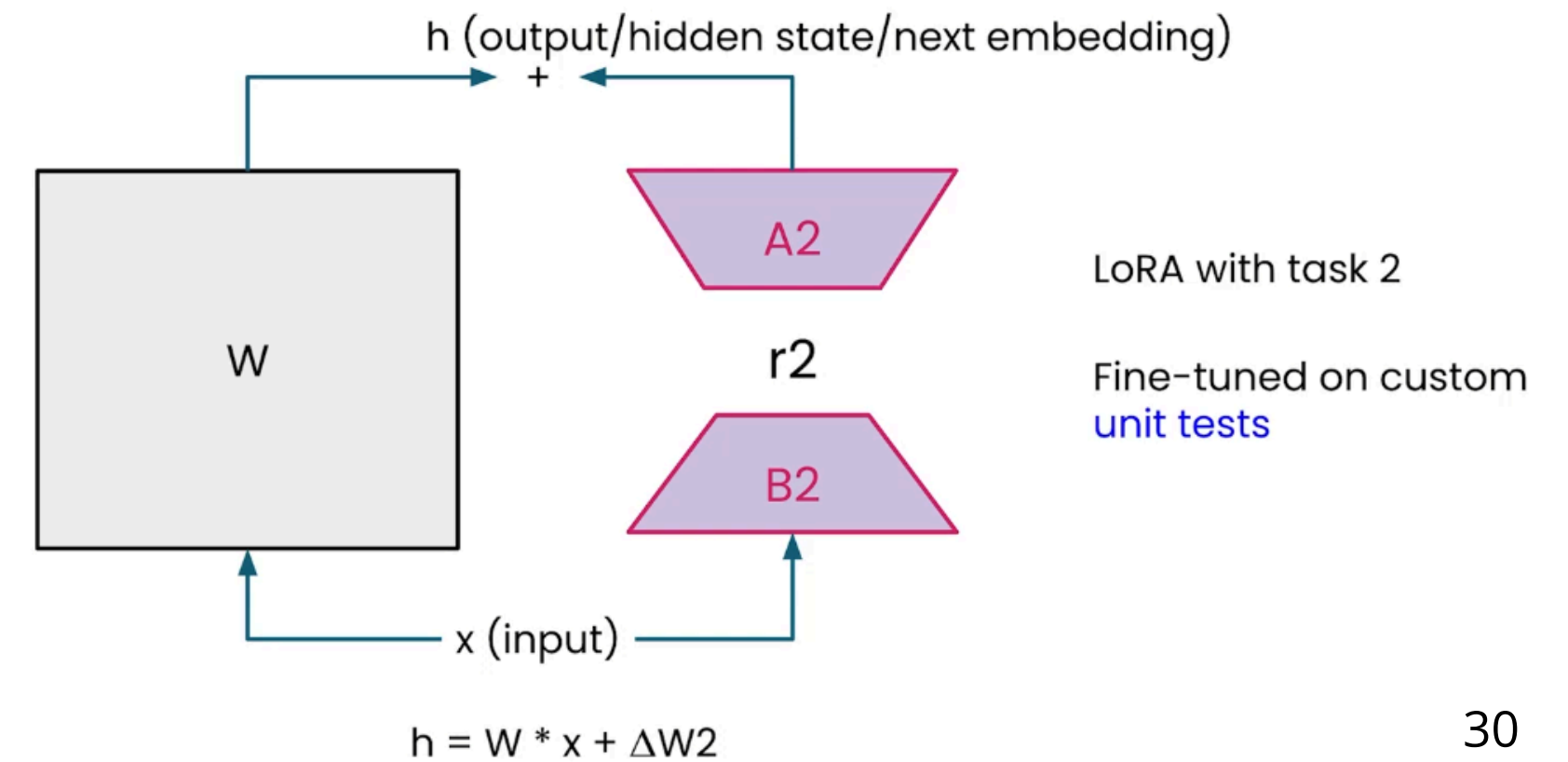
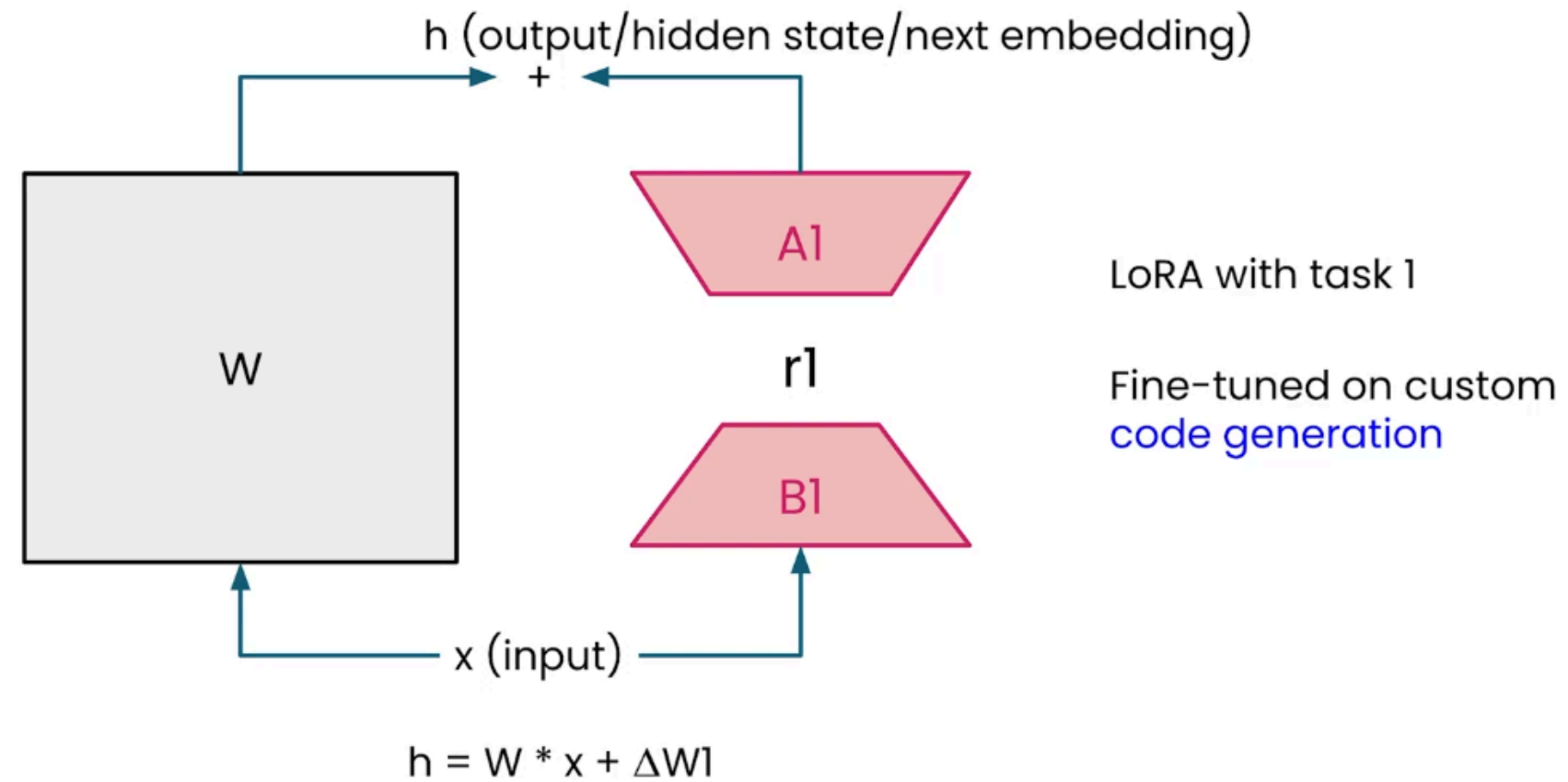
LoRA — Low-Rank Adaptation





Parameter-Efficient Fine-Tuning (PEFT)

LoRA — Low-Rank Adaptation





Parameter-Efficient Fine-Tuning (PEFT)

LoRA — Low-Rank Adaptation

α scales how much LoRAs vs. the original $\mathbf{W}$ weights matter

$$\mathbf{W}_{FT} = \mathbf{W} + \frac{\alpha}{r} \Delta \mathbf{W}$$

** cases da CEMIG → todos os melhores resultados foram com:

```
lora_r: 16  
lora_alpha: 32
```

Parameter-Efficient Fine-Tuning (PEFT)

LoRA — Low-Rank Adaptation

Total Memory Usage (PyTorch) – Apply LoRA

LLM Fine-tuned

Total_memory = model_memory



LLM Fine-tuned (with LoRA)

Total_memory(Apply LoRA) = model_memory



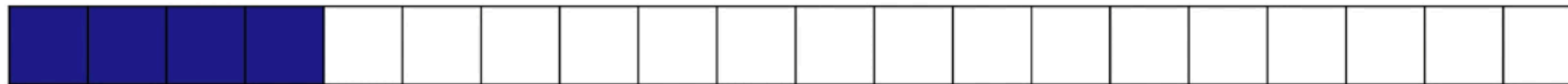


Parameter-Efficient Fine-Tuning (PEFT)

LoRA — Low-Rank Adaptation

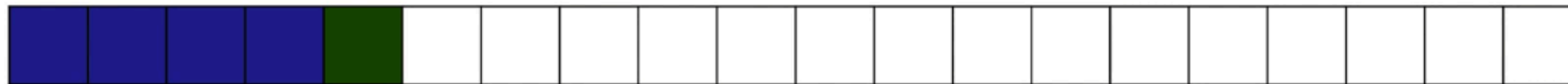
LLM Fine-tuned

Total_memory = model_memory



LLM Fine-tuned (with LoRA)

Total_memory(Apply LoRA) = model_memory + LoRA Memory (Trainable Parameters)



Trainable parameters
can be as small as
0.1%

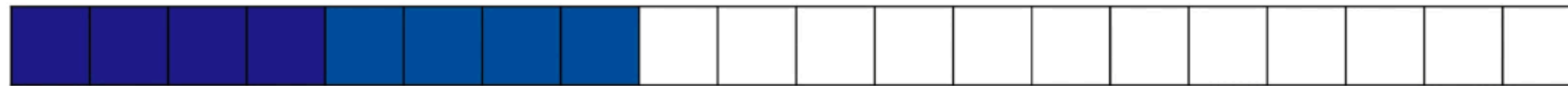


Parameter-Efficient Fine-Tuning (PEFT)

LoRA — Low-Rank Adaptation

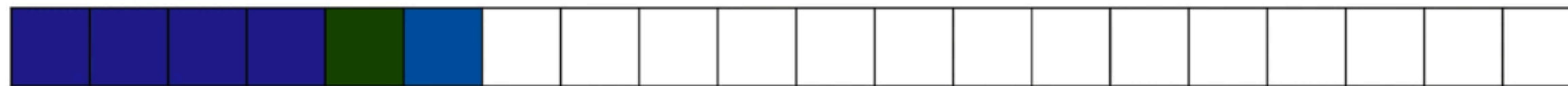
LLM Fine-tuned

Total_memory = model_memory + gradient_memory



LLM Fine-tuned (with LoRA)

Total_memory(Apply LoRA) = model_memory + LoRA Memory (Trainable Parameters) + gradient_memory
(Only consider trainable parameters)



Trainable parameters
can be as small as
0.1%



Parameter-Efficient Fine-Tuning (PEFT)

LoRA — Low-Rank Adaptation

LLM Fine-tuned

Total_memory = model_memory + gradient_memory + Optimization state (Depends on Optimizer, SGD:0, RMSProp:1, Adam:2) * gradient_memory



LLM Fine-tuned (with LoRA)

Total_memory(Apply LoRA) = model_memory + LoRA Memory (Trainable Parameters) + gradient_memory (Only consider trainable parameters) + Optimization state (Depends on Optimizer, SGD:0, RMSProp:1, Adam:2) * gradient_memory



Trainable parameters
can be as small as
0.1%

Only
consider
trainable
parameters



Parameter-Efficient Fine-Tuning (PEFT)

LoRA — Low-Rank Adaptation

LLM Fine-tuned

Total_memory = model_memory + gradient_memory + Optimization state o (Depends on Optimizer, SGD:0, RMSProp:1,Adam:2) *gradient_memory + forward_pass_memory



LLM Fine-tuned (with LoRA)

Total_memory(Apply LoRA) = model_memory + LoRA Memory (Trainable Parameters) + gradient_memory (Only consider trainable parameters) + Optimization state o (Depends on Optimizer, SGD:0, RMSProp:1, Adam:2) * gradient_memory + forward_pass_memory (Including LoRA)



Trainable parameters
can be as small as
0.1%

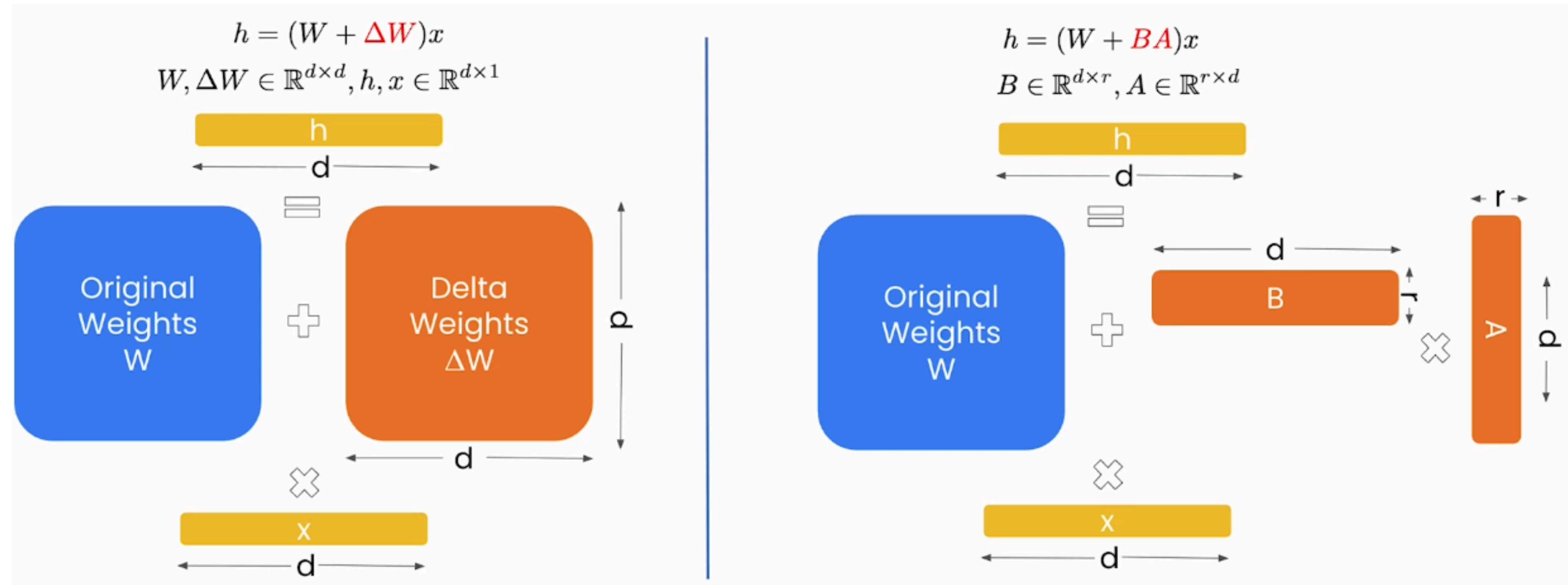
Only
consider
trainable
parameters

Extra LoRA compute
if not merged into
base.



Full SFT x PEFT

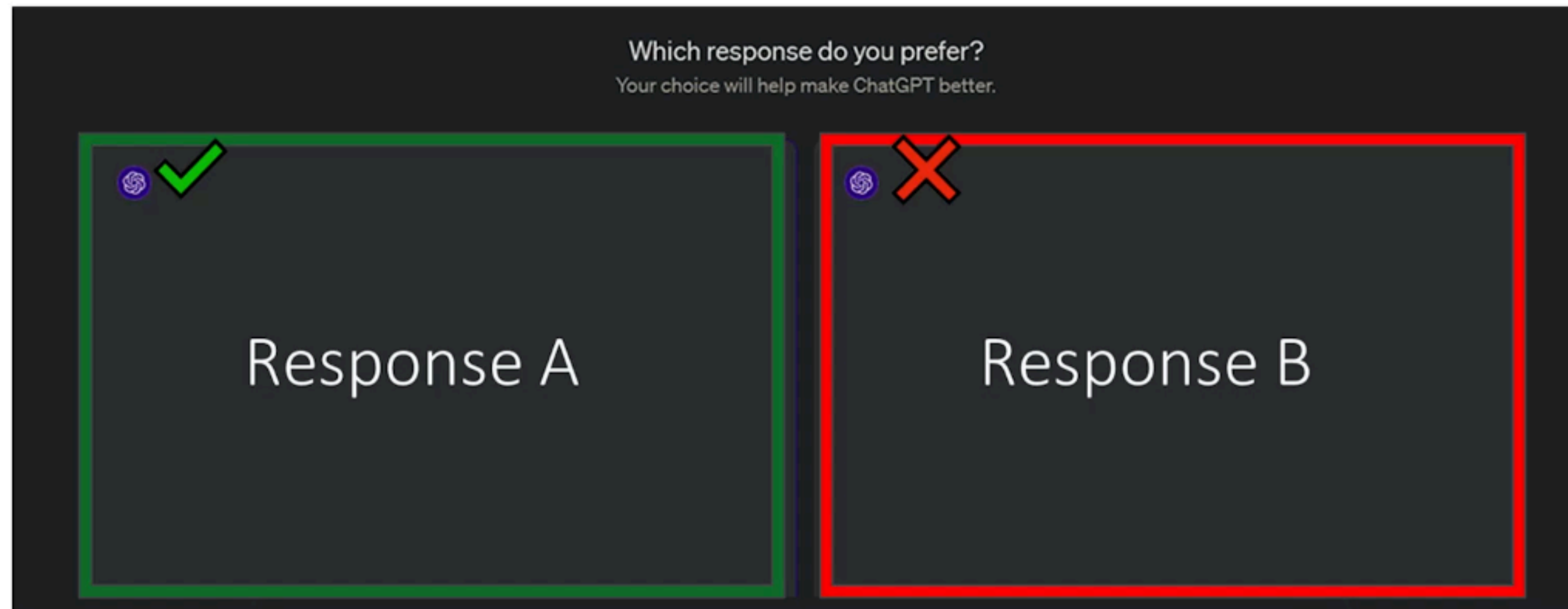
Like LoRA



Both full-finetuning and PEFT can be used in any of the post-training methods.
PEFT like Lora saves memory, learns less while forgets less [1]



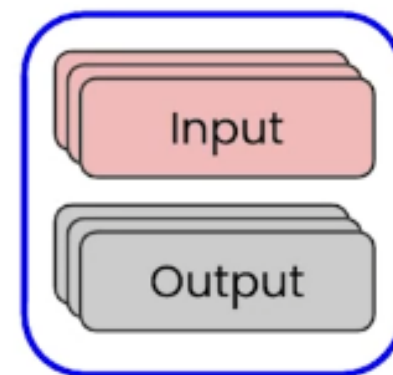
Reinforcement Learning from Human Feedback (RLHF)





Reinforcement Learning from Human Feedback (RLHF)

Reward Modeling



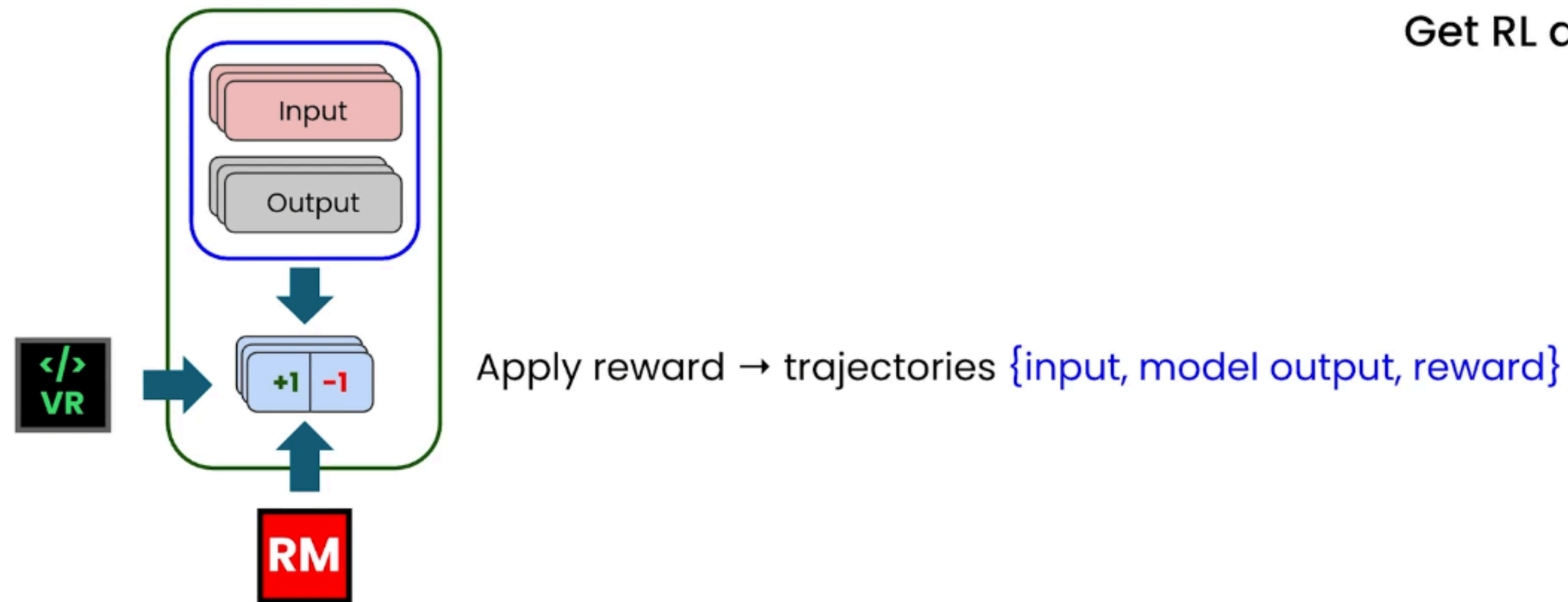
Get rollouts {input, model output}

Get RL data



Reinforcement Learning from Human Feedback (RLHF)

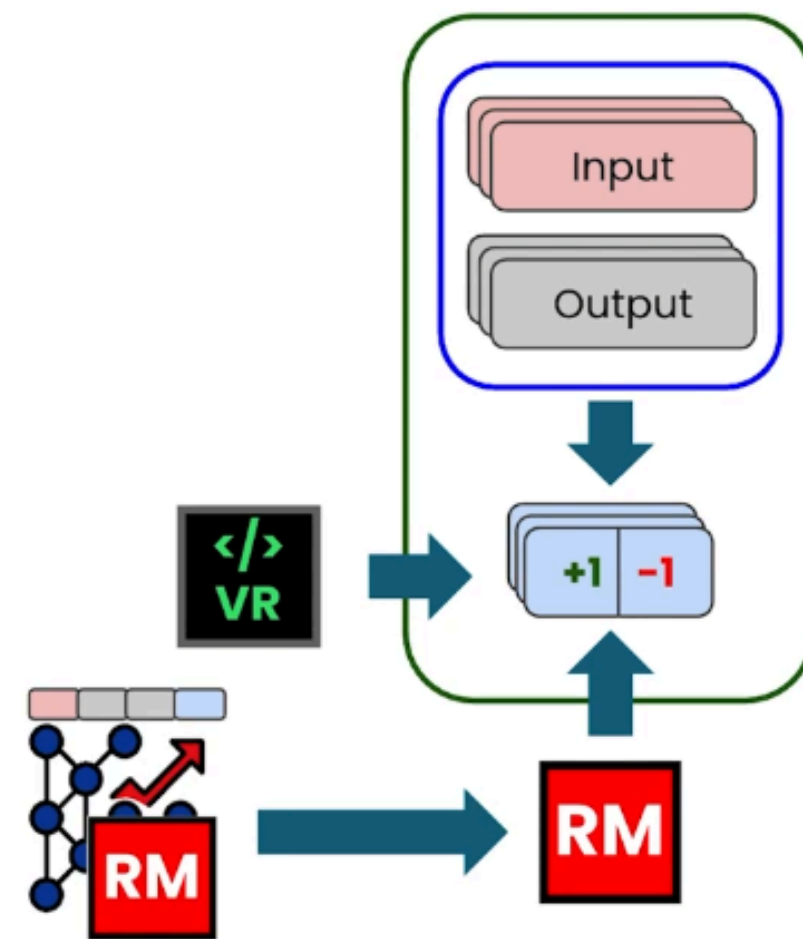
Reward Modeling





Reinforcement Learning from Human Feedback (RLHF)

Reward Modeling



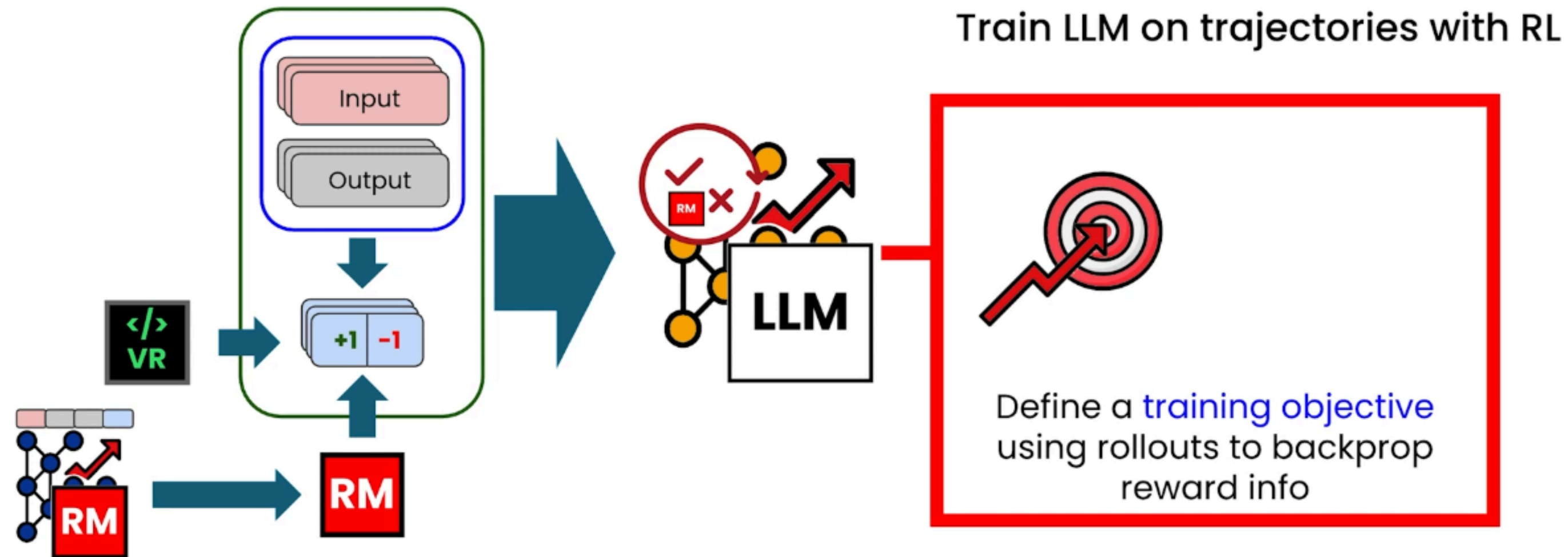
In **RLHF**, train a reward model with preference learning

Get RL data



Reinforcement Learning from Human Feedback (RLHF)

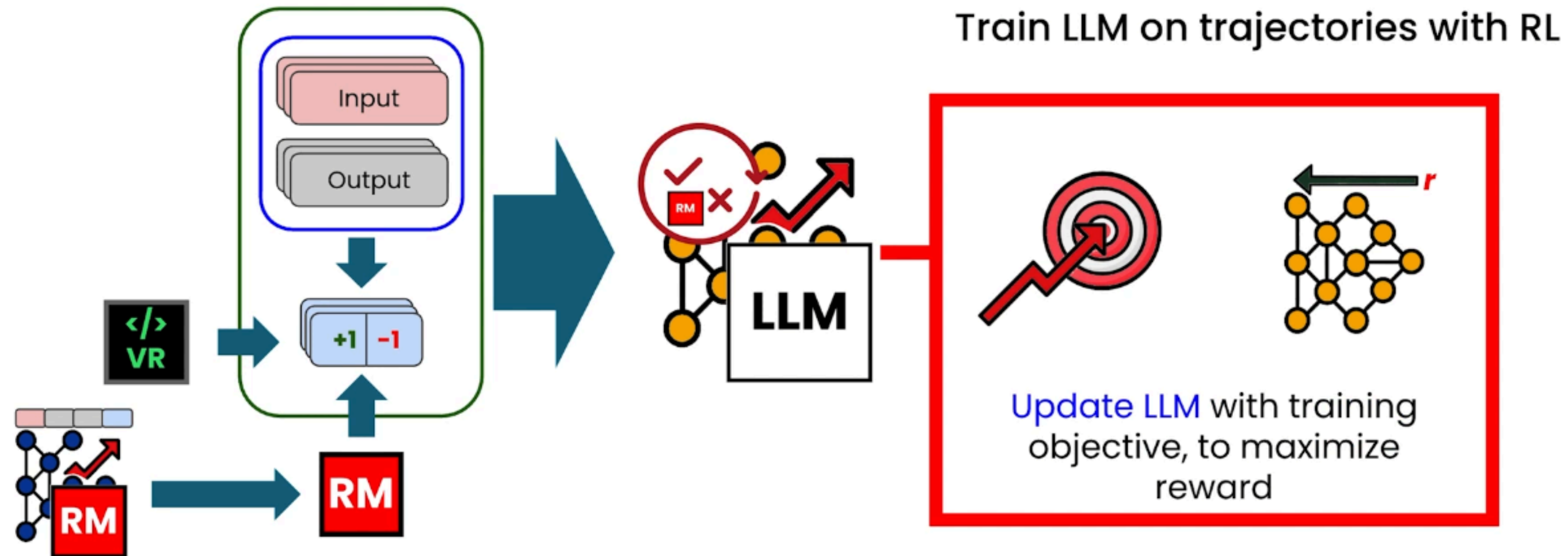
Reward Modeling





Reinforcement Learning from Human Feedback (RLHF)

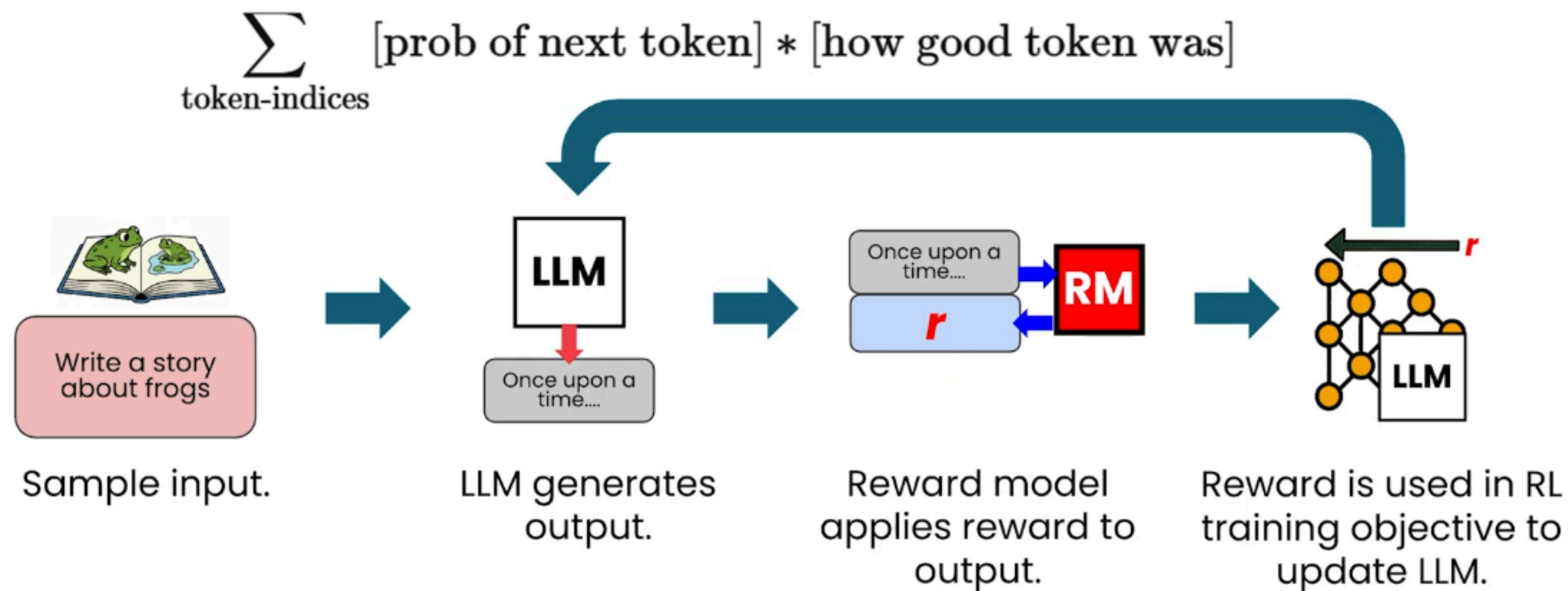
Reward Modeling





Reinforcement Learning from Human Feedback (RLHF)

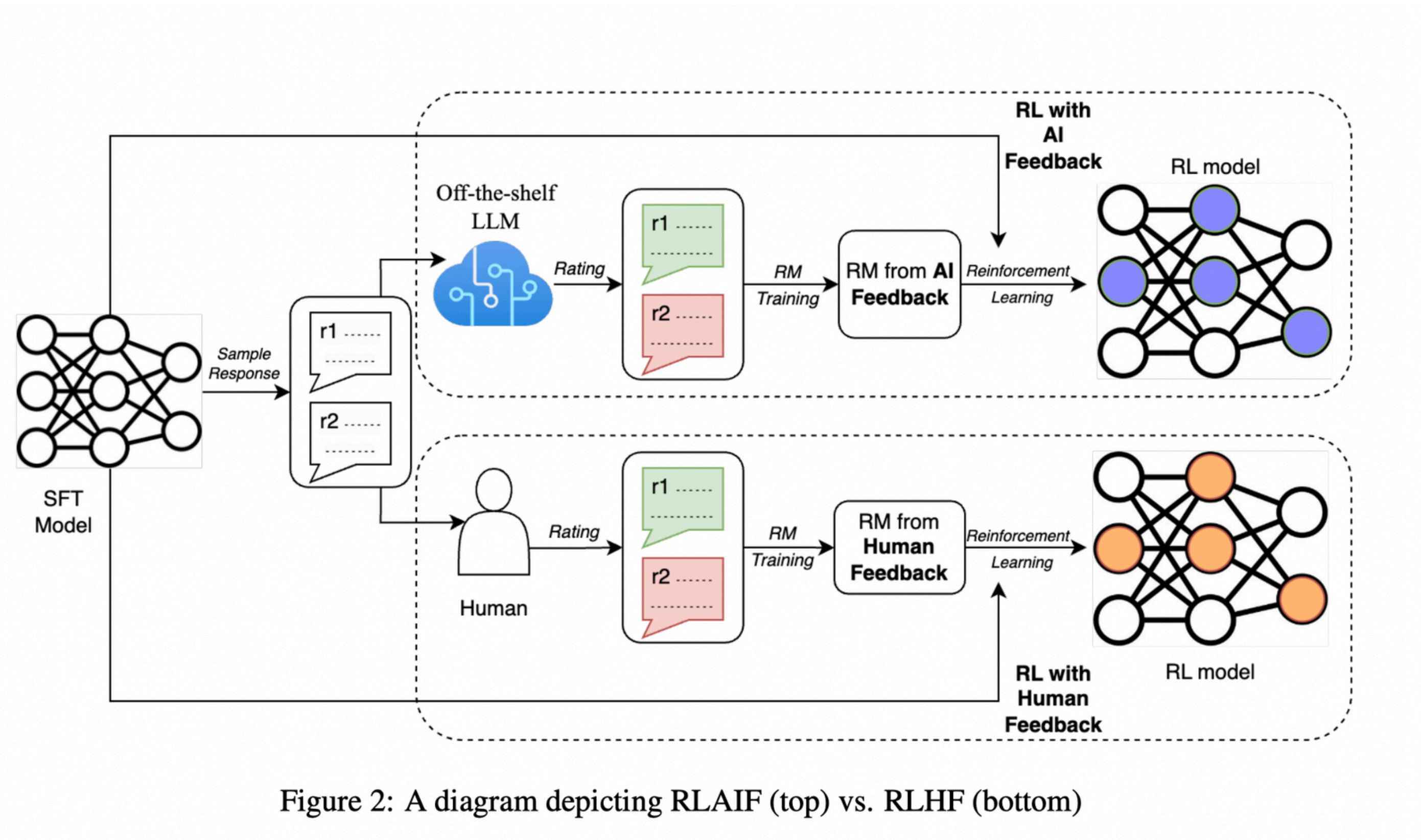
Reward Modeling





Reinforcement Learning from Human Feedback (RLHF)

Human Feedback vs. AI Feedback





Proximal Policy Optimization (PPO)

Policy Objective

$$\sum_{\text{token-indices}} [\text{prob of next token}] * [\text{how good token was}]$$

- We have *almost* arrived at our **first modern RL algorithm**.
- Our probability ratio sometimes gets **too large**. We will adjust our objective to **clip** this ratio!

$$\sum_{\text{token-indices}} \min \left(\frac{\text{LLM}(y_i | \mathbf{x}, \mathbf{y}_{<i})}{\text{LLM}_{\text{ref}}(y_i | \mathbf{x}, \mathbf{y}_{<i})} * (r(y_i, \mathbf{x}, \mathbf{y}_{<i}) - \text{LLM}_2(\mathbf{x}, \mathbf{y}_{<i})), \text{clip} \left(\frac{\text{LLM}(y_i | \mathbf{x}, \mathbf{y}_{<i})}{\text{LLM}_{\text{ref}}(y_i | \mathbf{x}, \mathbf{y}_{<i})}, \epsilon, 1 - \epsilon \right) * (r(y_i, \mathbf{x}, \mathbf{y}_{<i}) - \text{LLM}_2(\mathbf{x}, \mathbf{y}_{<i})) \right)$$

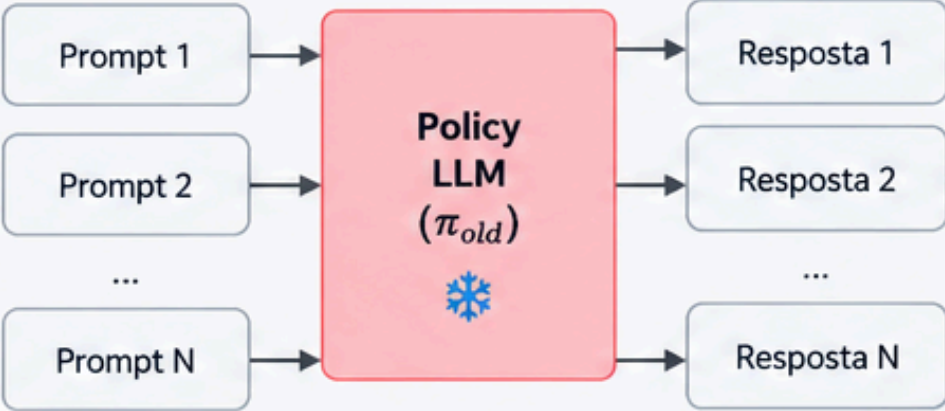
This is PPO!

PPO for LLMs: o que é treinável?

Visão geral do pipeline, mostrando quais componentes são atualizados e onde passa o backpropagation.

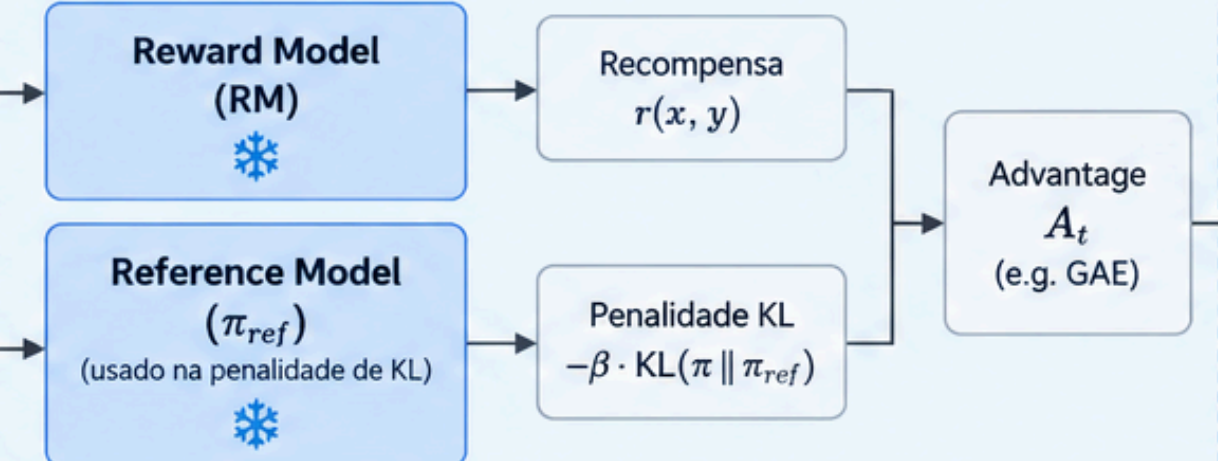
Coleta de dados (rollout)

(sem gradiente)



Cálculo de recompensa e advantage

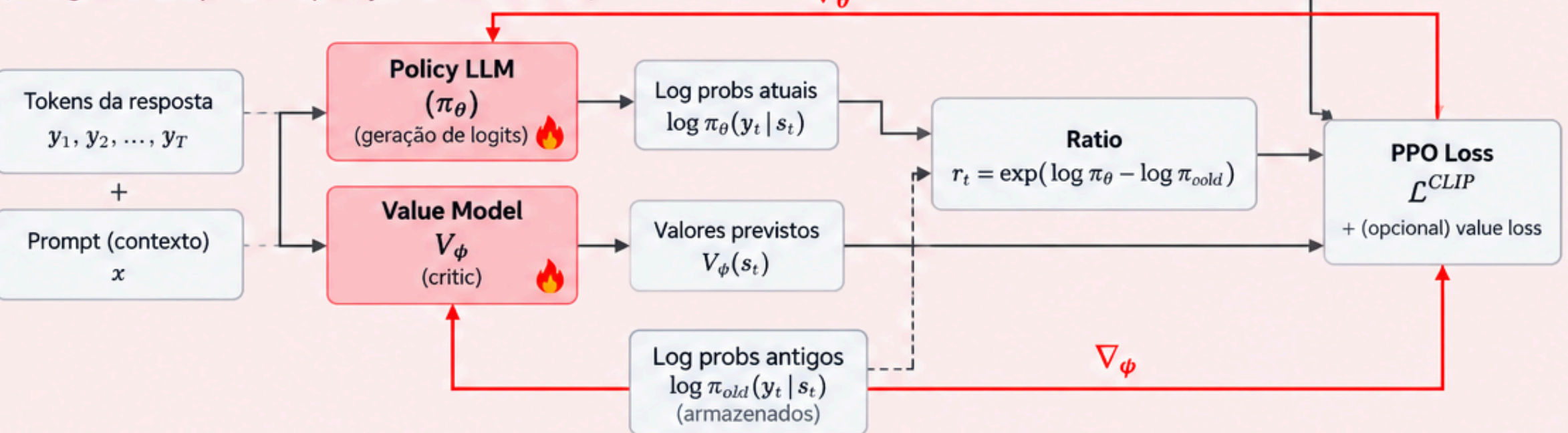
(sem gradiente)



- Treinável (atualizado por gradiente)
- Congelado (não treinável)
- Passa gradiente (backpropagation)
- Sem gradiente

Otimização (PPO)

(com gradiente apenas na policy e no value model)



- **Policy LLM (π_θ):** treinável. Recebe gradiente da PPO loss e é atualizada.
- **Value Model (V_ϕ):** treinável. Recebe gradiente da value loss (ex.: MSE para o retorno).
- **Reward Model (RM):** não treinável. Usado apenas para gerar a recompensa.
- **Reference Model (π_{ref}):** não treinável. Usado na penalidade de KL.
- **Old Policy (π_{old}):** não treinável. É uma cópia da policy no momento da coleta de dados (ou log probs armazenados).
- **Gradiente** não passa por RM, reference model, nem pela old policy.

PPO (objetivo da policy)

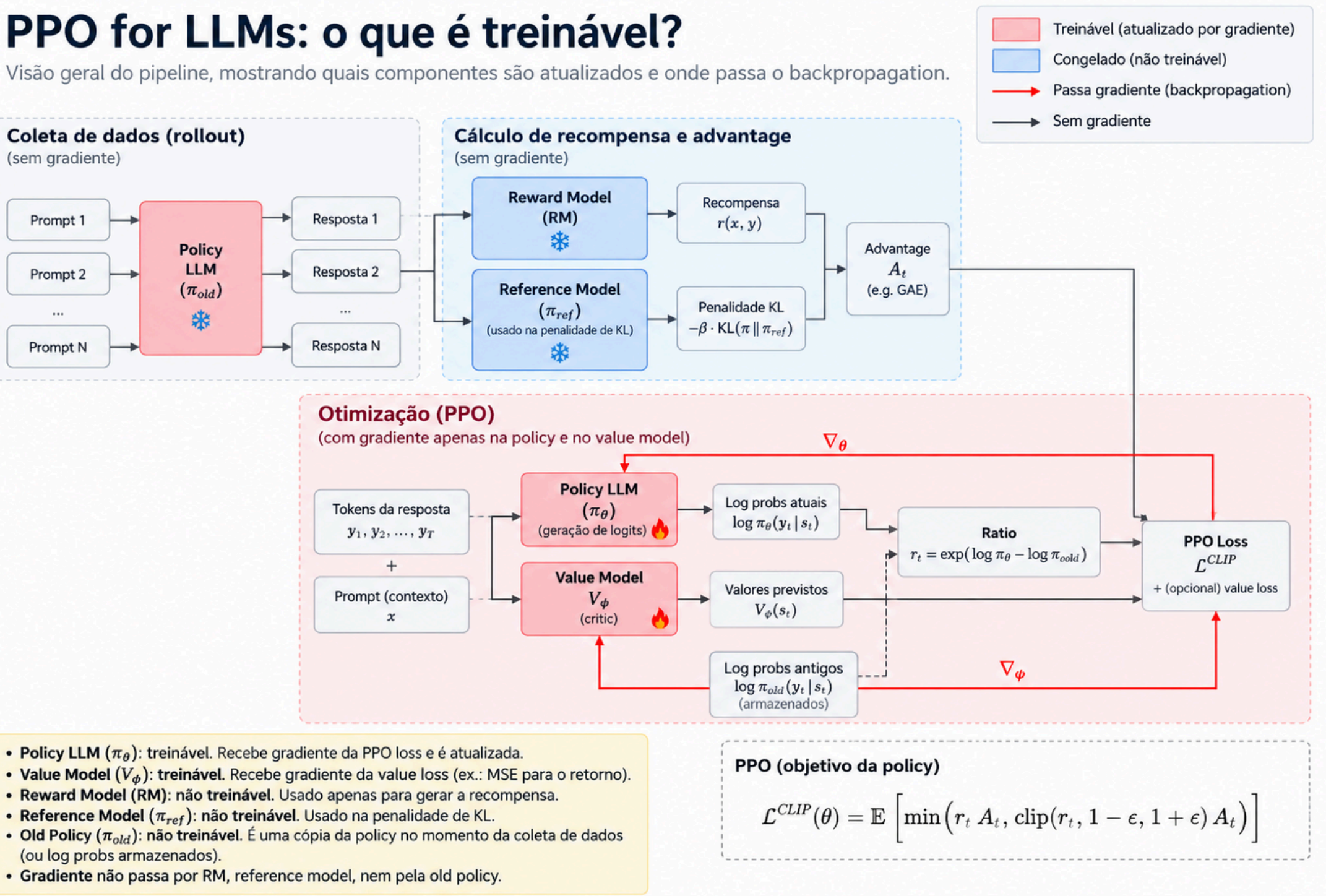
$$\mathcal{L}^{CLIP}(\theta) = \mathbb{E} \left[\min \left(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t \right) \right]$$

Qwen3-4B base→SFT energia→PPO

- Policy atual (π_θ) = Qwen3-4B-SFT-Energia
- π_{old} = Qwen3-4B-PPO-v17
- V_ϕ = Qwen3-4B + Value Head
- RM = Outro LLM

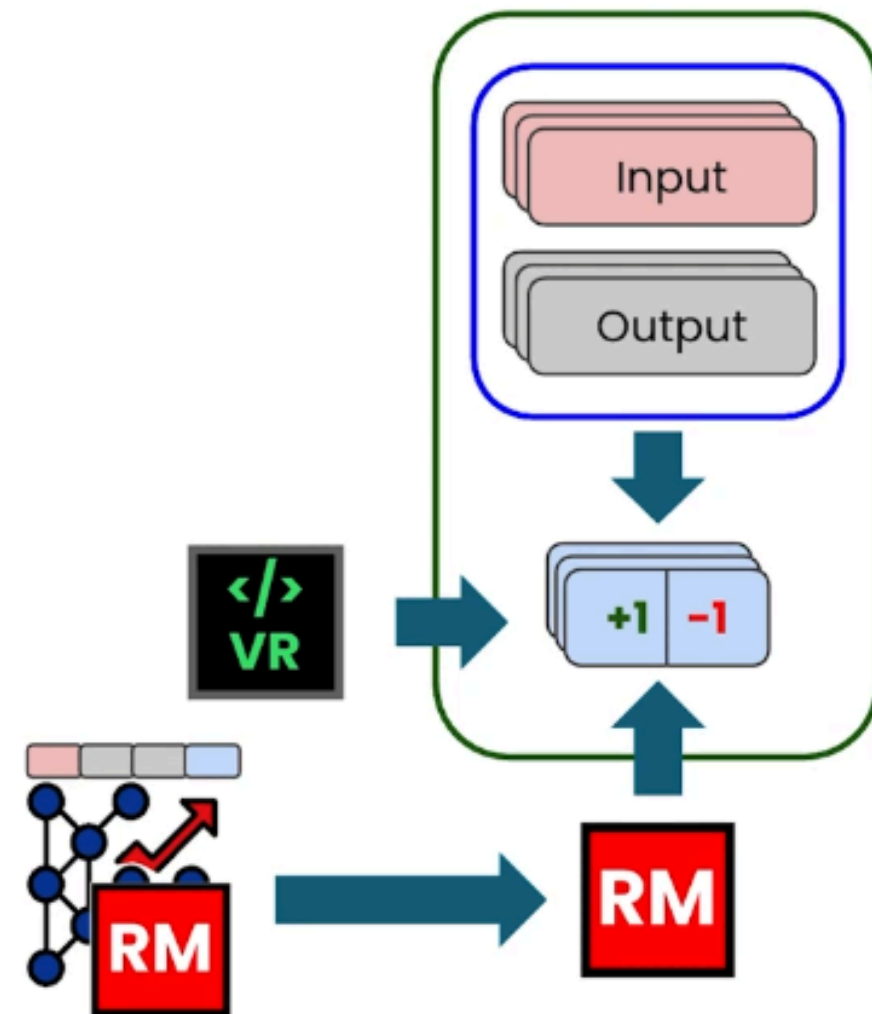
PPO for LLMs: o que é treinável?

Visão geral do pipeline, mostrando quais componentes são atualizados e onde passa o backpropagation.





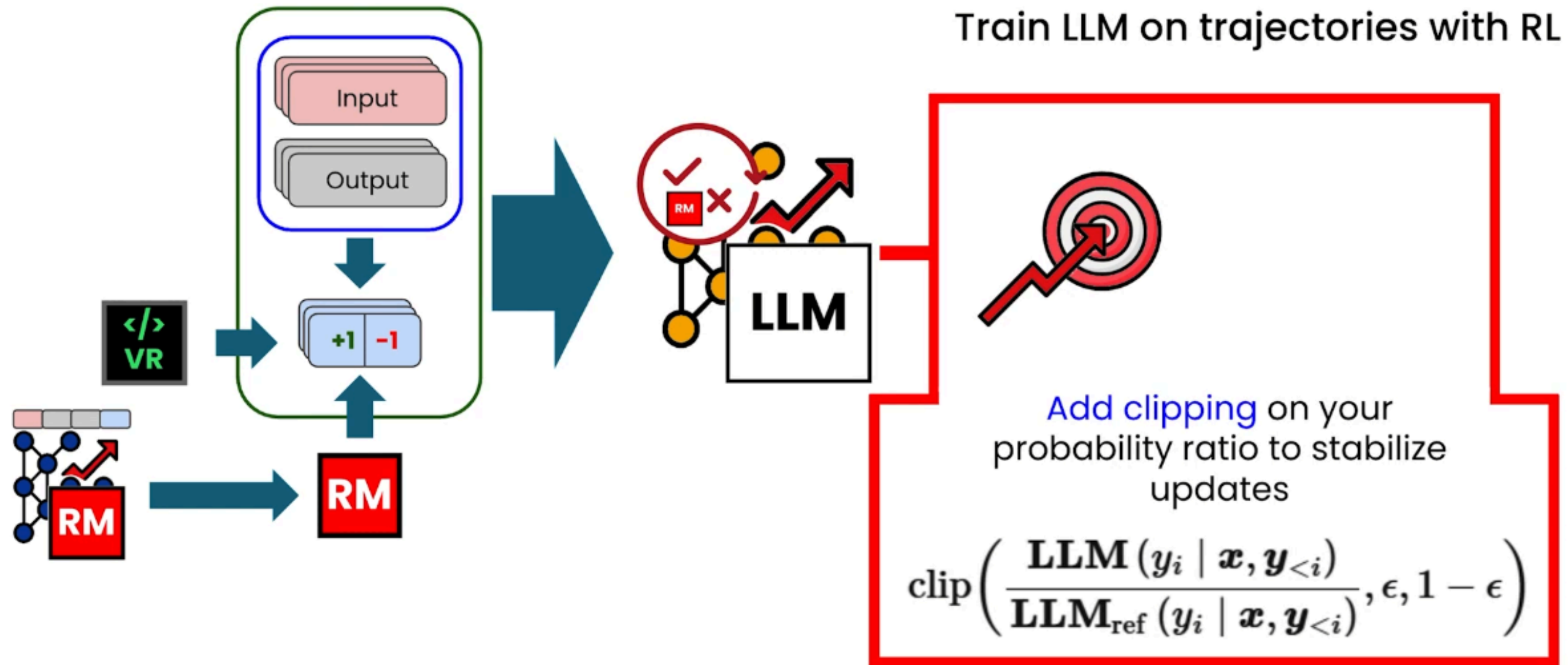
Proximal Policy Optimization (PPO)



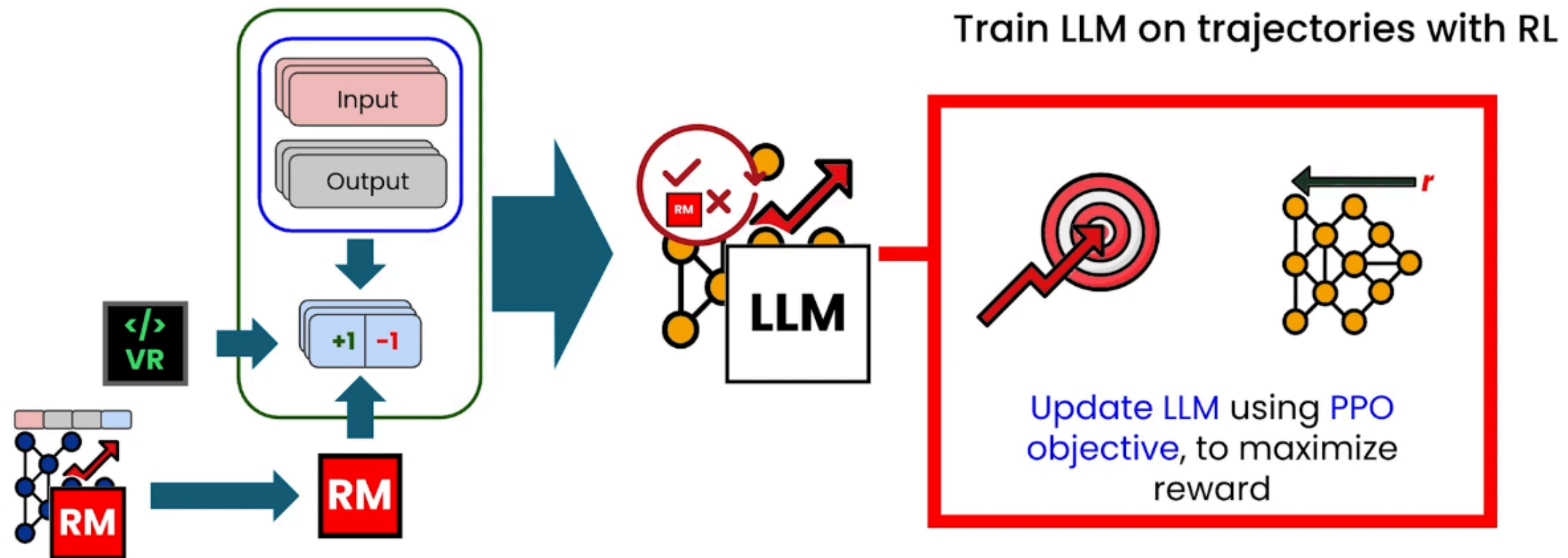
Get RL data
(as usual)



Proximal Policy Optimization (PPO)

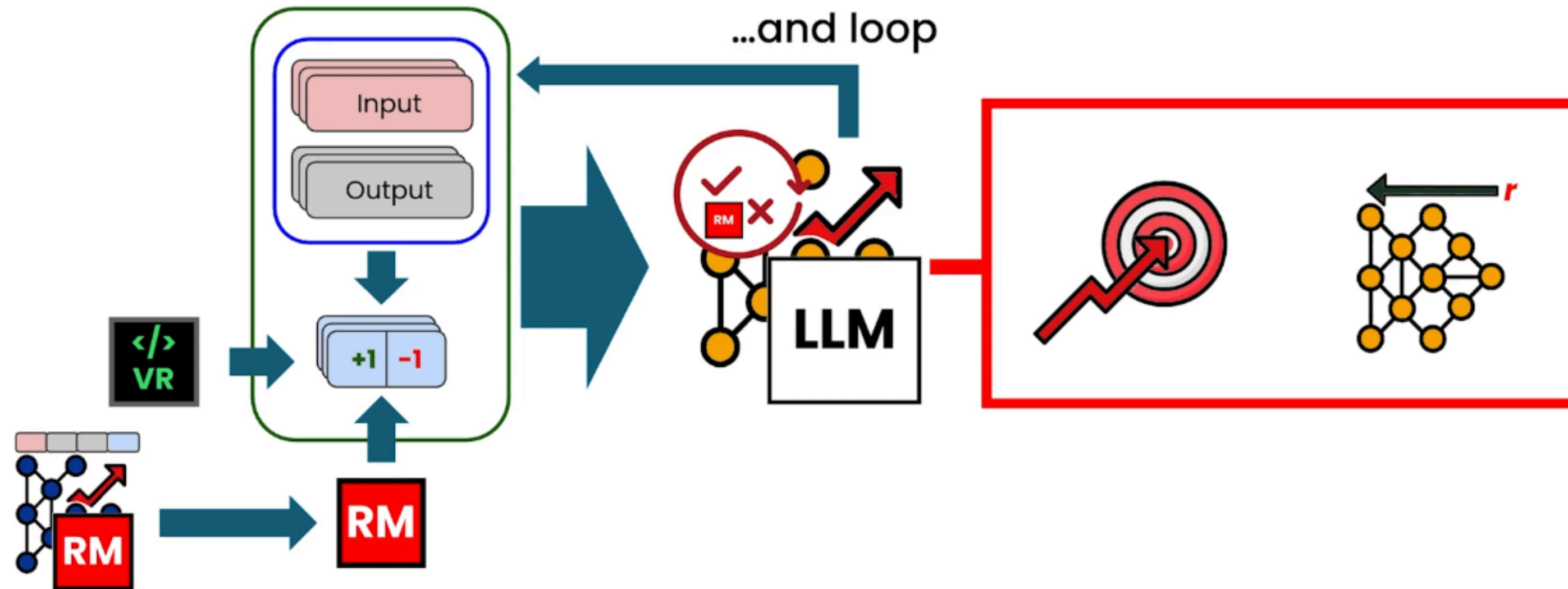


Proximal Policy Optimization (PPO)





Proximal Policy Optimization (PPO)





Direct Preference Optimization (DPO)

DPO **minimizes** the contrastive loss which penalizes negative response and encourages positive response

DPO loss is a cross entropy loss on the reward difference of a “re-parameterized” reward model

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left(\log \frac{\pi_{\theta}(y_{\text{pos}} | x)}{\pi_{\text{ref}}(y_{\text{pos}} | x)} - \left[\log \frac{\pi_{\theta}(y_{\text{neg}} | x)}{\pi_{\text{ref}}(y_{\text{neg}} | x)} \right] \right) \right)$$

Diagram illustrating the DPO loss formula with annotations:

- Sigmoid function**: Points to the σ function in the formula.
- Fine-tuned model**: Points to the π_{θ} terms in the formula.
- hyperparameter**: Points to the β parameter in the formula.
- Reference model (copy of the original model)**: Points to the π_{ref} terms in the formula.
- Reparameterization of reward model**: Points to the dashed red box around the negative log ratio term, indicating it is the reparameterized reward model.

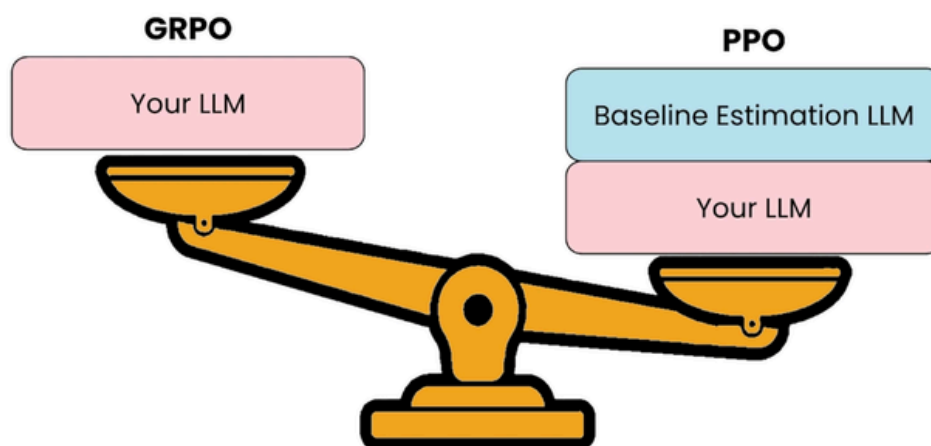


Group Relative Policy Optimization (GRPO)

"GRPO forgoes the critic model [aka. separate baseline estimation model], instead estimating the baseline from group scores, significantly reducing training resources."

– DeepSeekMath paper

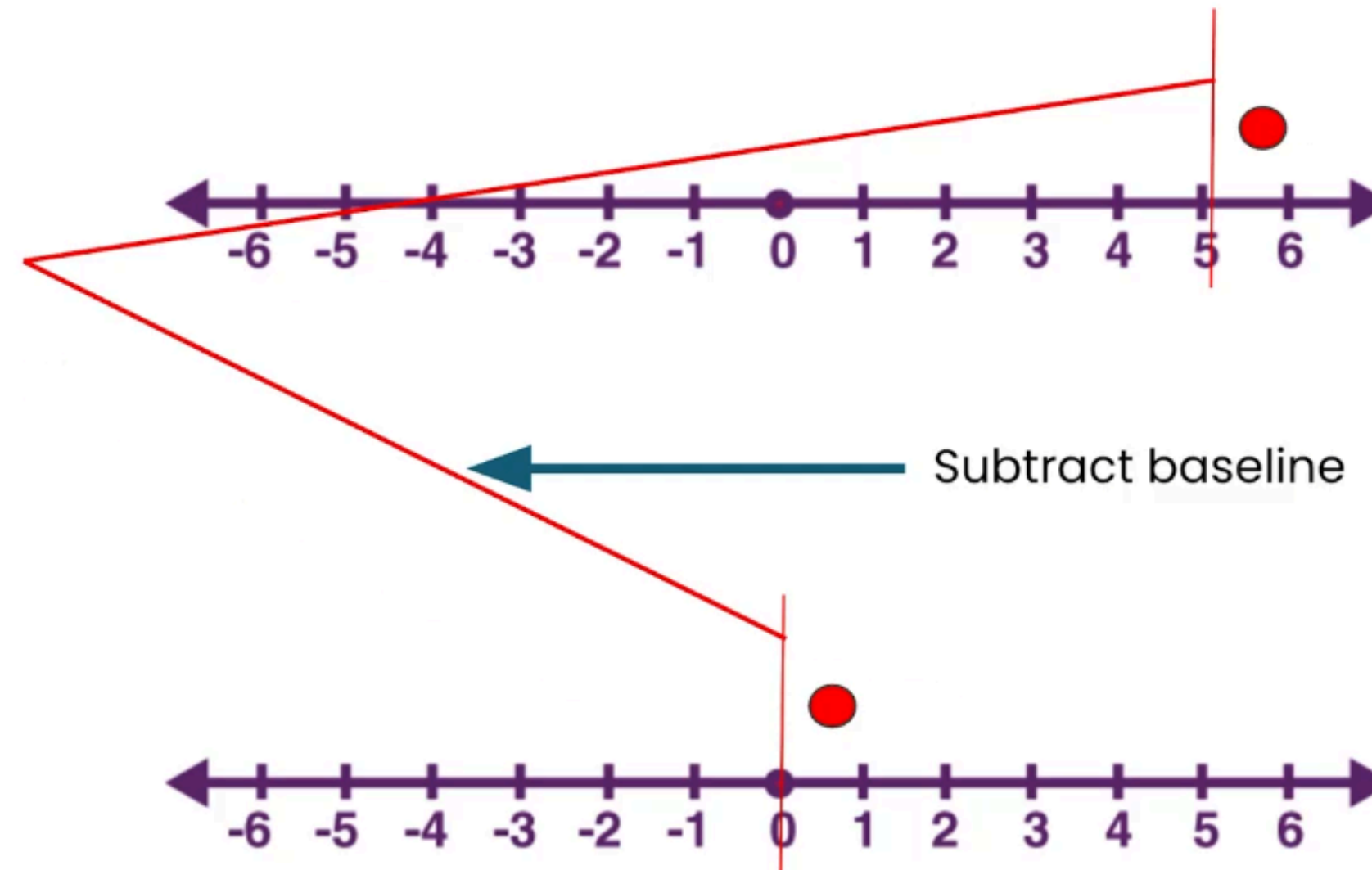
[From "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models", Shao et al., 2024]





Group Relative Policy Optimization (GRPO)

Instead of training a **new model** to compute the baseline, why not just...



GRPO

Your LLM

PPO

Baseline Estimation LLM

Your LLM



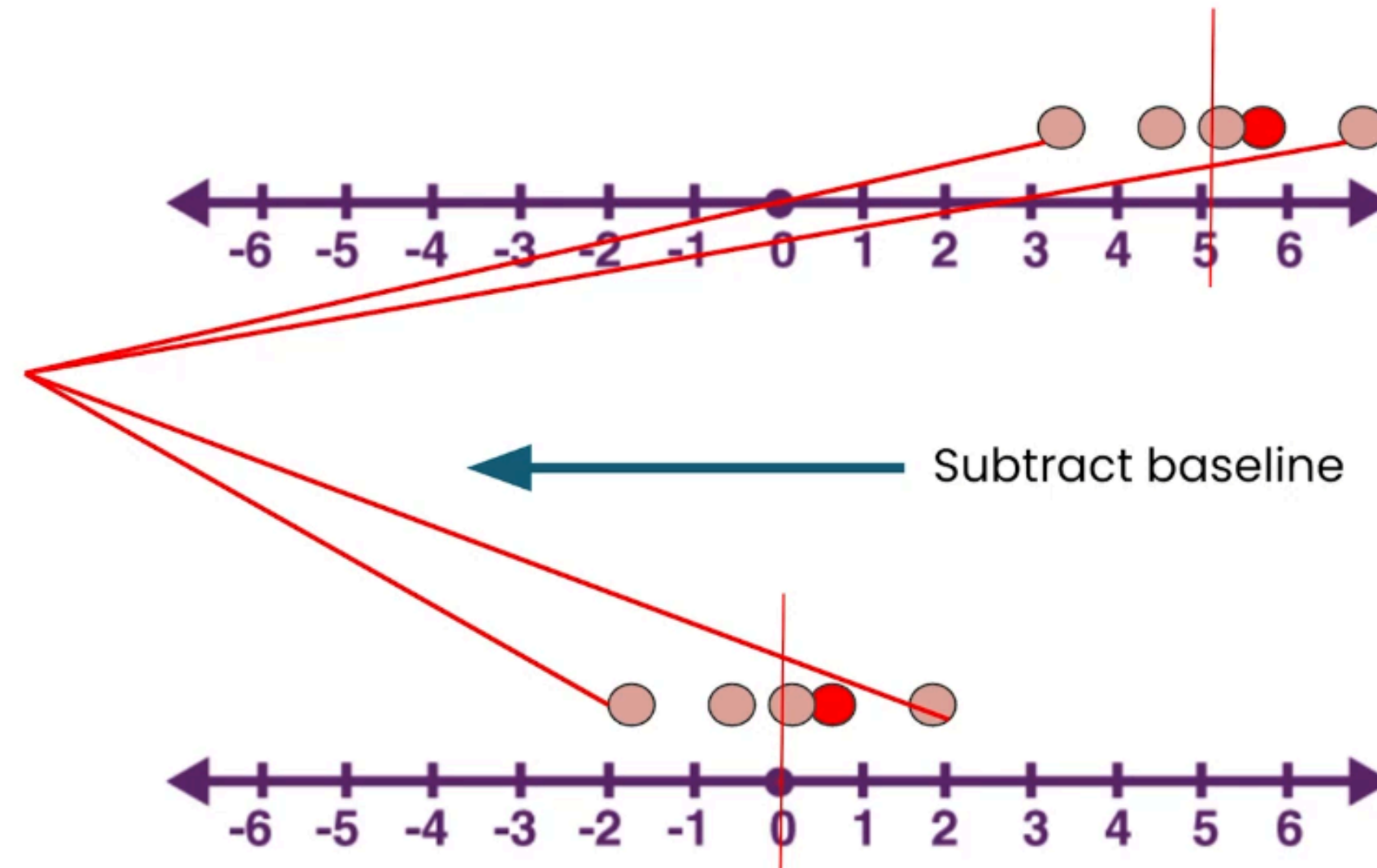


Group Relative Policy Optimization (GRPO)

Sample **many outputs** per input.

These are called a **"group"**.

Average the rewards in a group.



GRPO

Your LLM

PPO

Baseline Estimation LLM

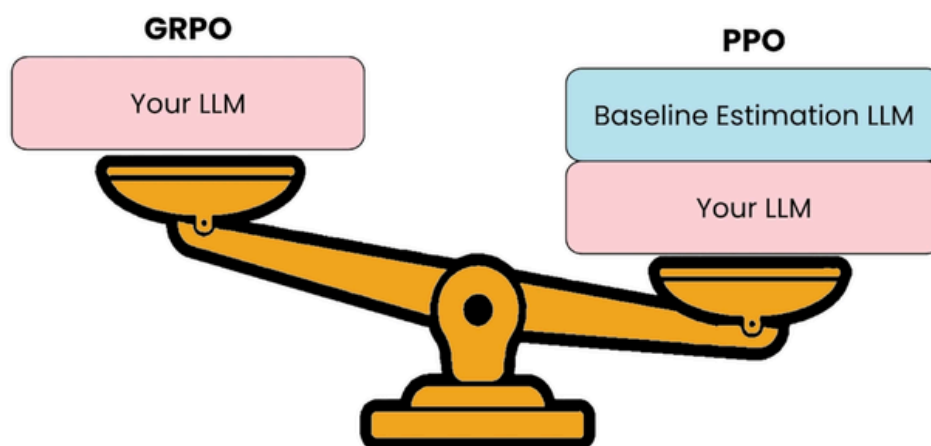
Your LLM



Group Relative Policy Optimization (GRPO)

$$\sum_{\text{token-indices}} [\text{prob of next token}] * [\text{how good token was}]$$

$$\frac{r(y_i, \mathbf{x}, \mathbf{y}_{<i}) - \text{mean}(r\text{'s in same group})}{\text{std}(r\text{'s in same group})}$$



Temperature helps you sample different outputs, per input.

```
GRPO_config = GRPOConfig(
    num_generations = 12
    temperature = 0.7)
```

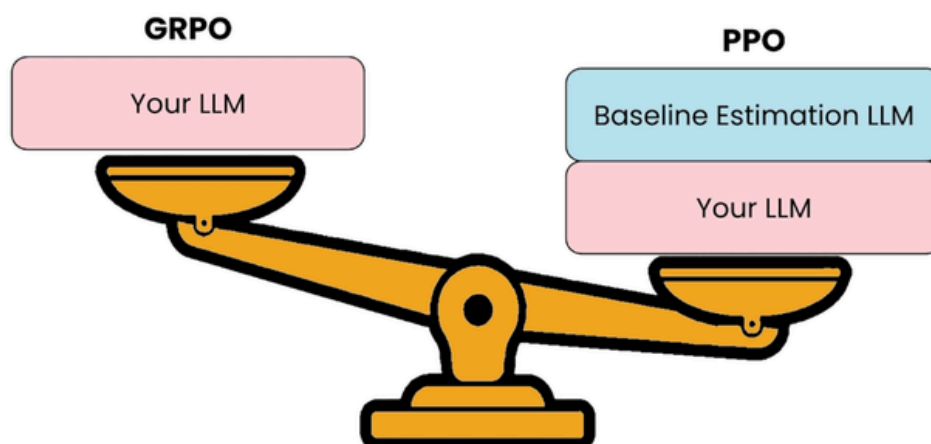


Group Relative Policy Optimization (GRPO)

$$\sum_{\text{token-indices}} [\text{prob of next token}] * [\text{how good token was}]$$

$$\frac{r(y_i, \mathbf{x}, \mathbf{y}_{<i}) - \text{mean}(r\text{'s in same group})}{\text{std}(r\text{'s in same group})}$$

```
mean_reward = sum(group_rewards) / len(group_rewards)
normalized_rewards = [r - mean_reward for r in group_rewards]
```

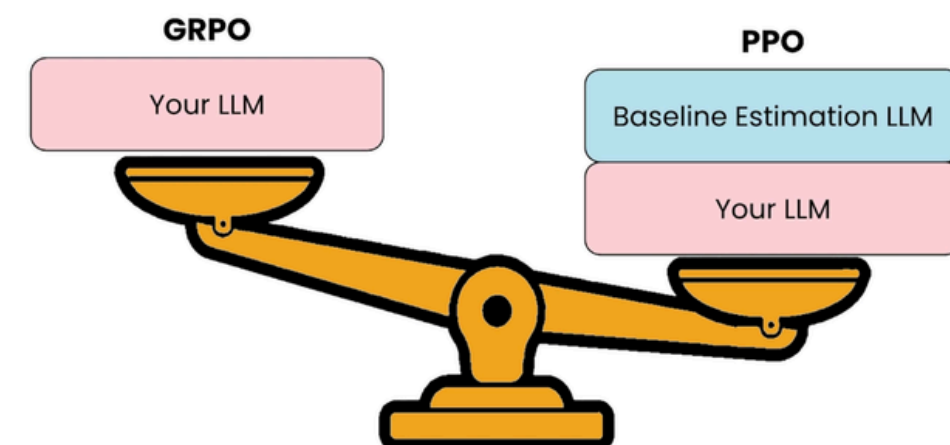




Group Relative Policy Optimization (GRPO)

Response	Raw Reward
A	8.5
B	6.8
C	4.2
D	2.5

Sample Grader
 Python grader
 Unit test grader
 Formatting grader

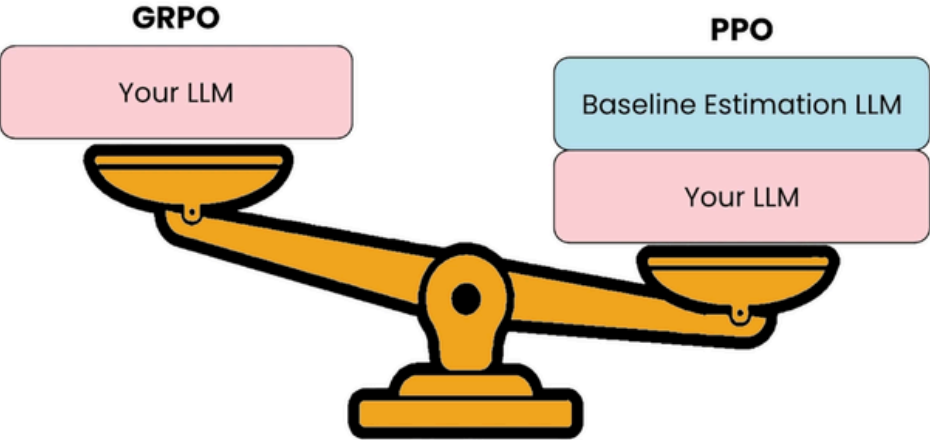




Group Relative Policy Optimization (GRPO)

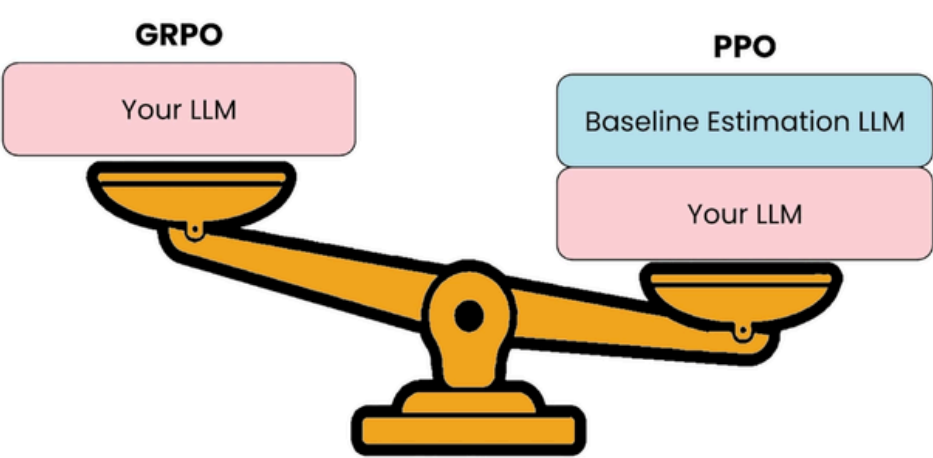
Response	Raw Reward	Advantage
A	8.5	+1.12
B	6.8	+0.49
C	4.2	-0.49
D	2.5	-1.12

$$\frac{r(y_i, \boldsymbol{x}, \boldsymbol{y}_{<i}) - \text{mean}(\text{r's in same group})}{\text{std}(\text{r's in same group})}$$



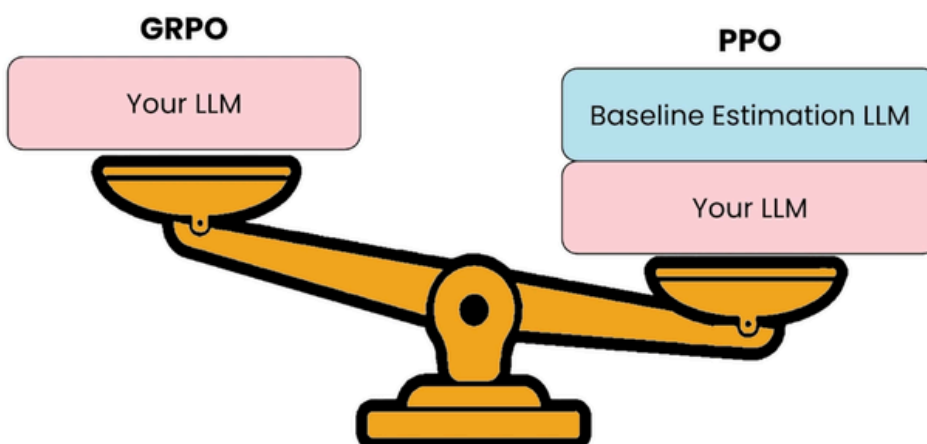
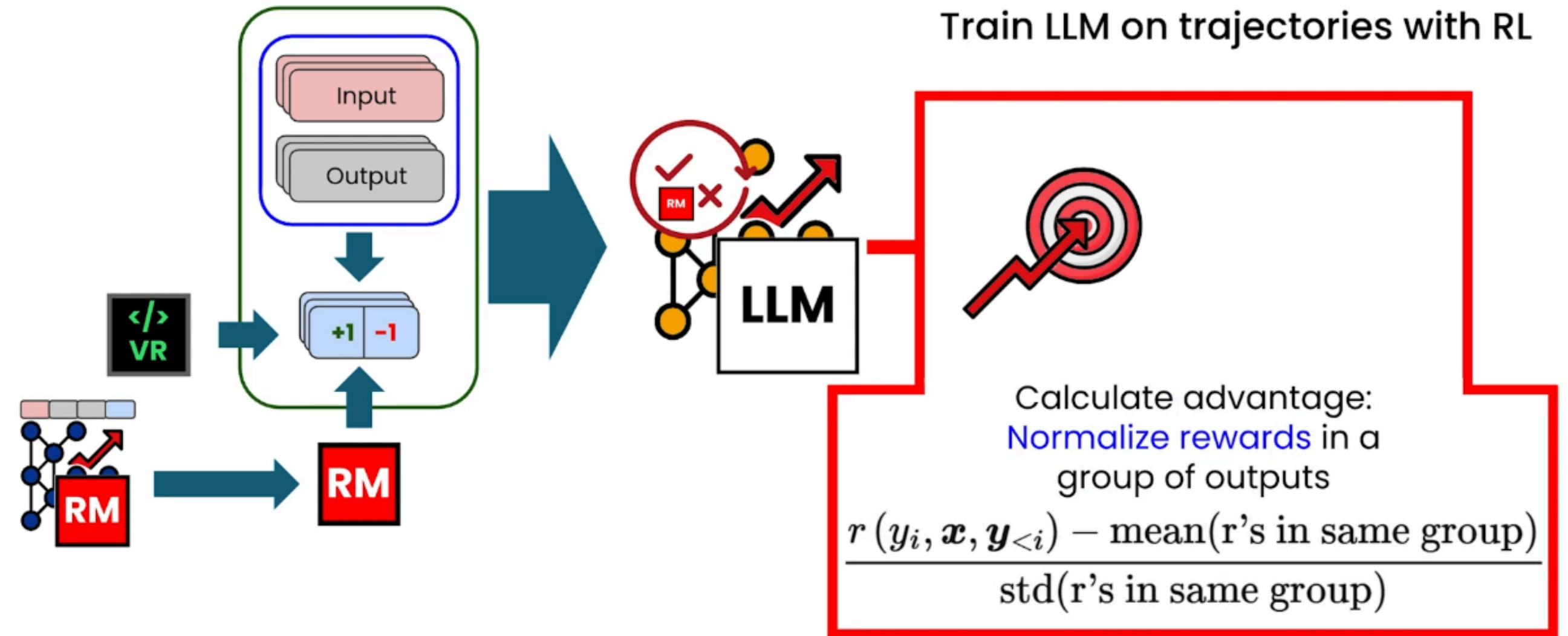
Group Relative Policy Optimization (GRPO)

Response	Raw Reward	Advantage	Effect
A	8.5	+1.12	Increase
B	6.8	+0.49	Increase
C	4.2	-0.49	Decrease
D	2.5	-1.12	Decrease





Group Relative Policy Optimization (GRPO)





Group Relative Policy Optimization (GRPO)

1. Your LLM – learning!

$$\mathbf{LLM}(y_i \mid \mathbf{x}, \mathbf{y}_{<i})$$

2. $\mathbf{LLM}_{\text{ref}}$ – frozen (inference only),
generates rollouts

$$\mathbf{LLM}_{\text{ref}}(y_i \mid \mathbf{x}, \mathbf{y}_{<i})$$

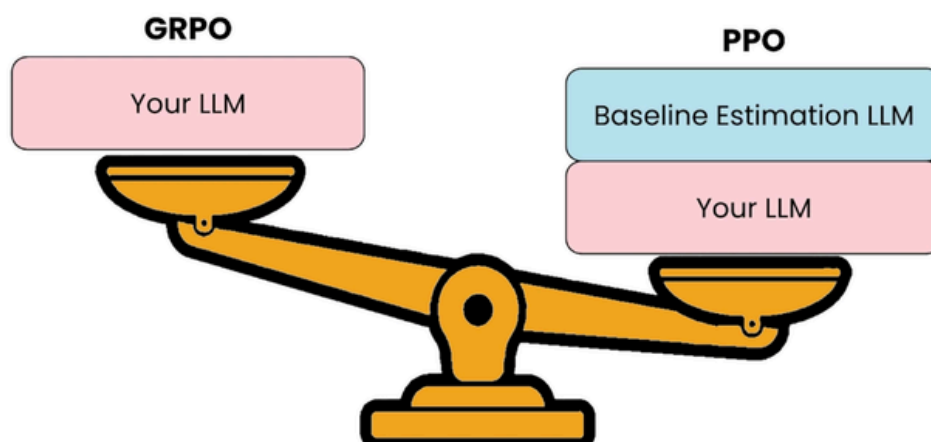
3. Reward Model

$$r(y_i, \mathbf{x}, \mathbf{y}_{<i})$$

4. Baseline Estimation LLM

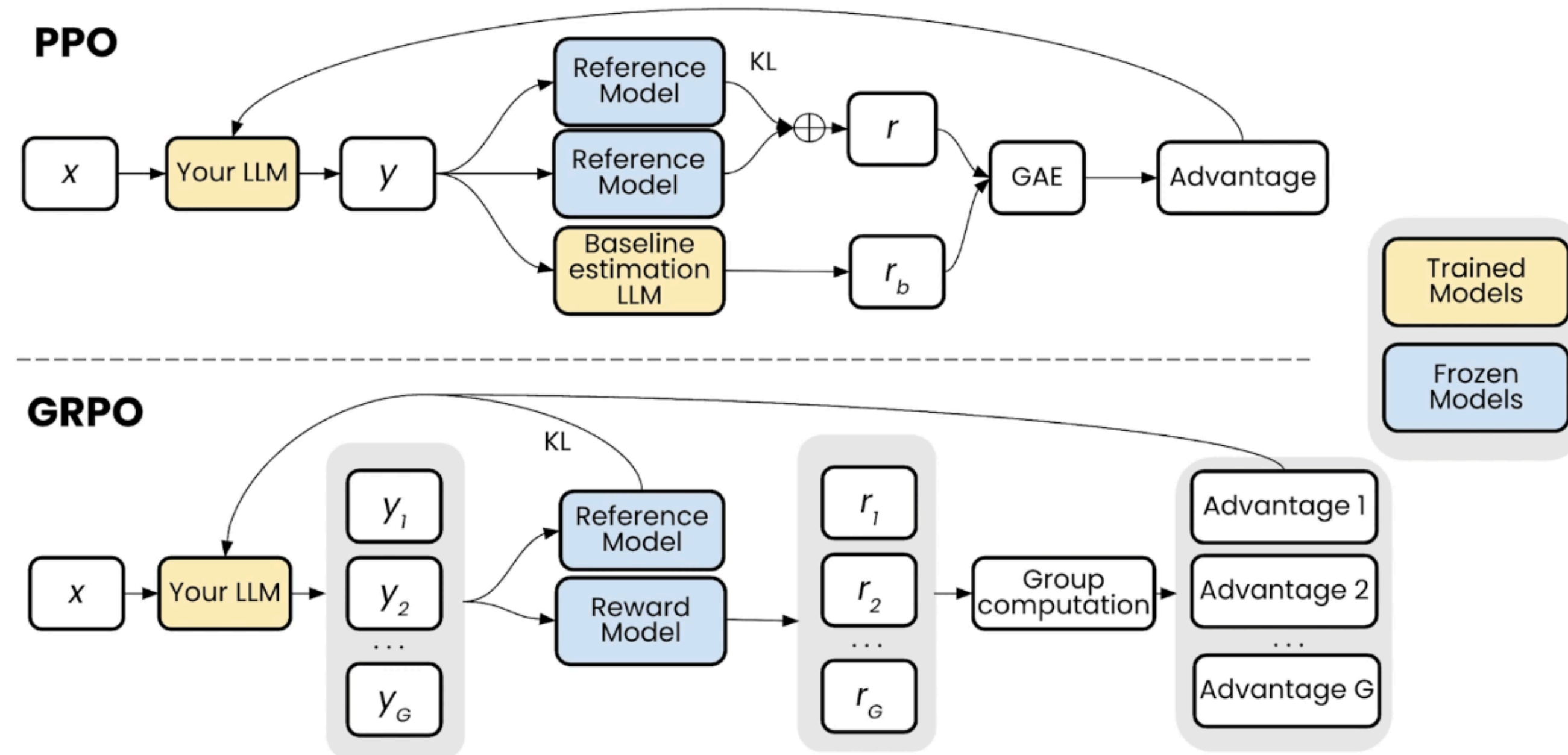
$$b(\mathbf{x}, \mathbf{y}_{<i})$$

PPO, not GRPO





Group Relative Policy Optimization (GRPO)



DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models <https://arxiv.org/abs/2402.03300>

